



TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

BÁO CÁO CUỐI KỲ NHẬP MÔN TRÍ TUỆ NHÂN TẠO

GVPT: ThS. Nguyễn Quốc Khoa



Nhóm: 10

MSSV	Họ và tên	Email
24200087	Huỳnh Ngọc Huy	24200087@student.hcmus.edu.vn
24200032	Choi Won Seok	24200032@student.hcmus.edu.vn

Mục lục

1	Giới Thiệu	5
1.1	Đặt Vấn Đề	5
1.2	Mục Tiêu Đề Án	5
1.3	Phương Pháp Tiếp Cận	5
2	Mô Tả Bộ Dữ Liệu	6
2.1	Tổng Quan về GTSRB	6
2.2	Cấu Trúc Thư Mục	6
2.3	Đặc Điểm Dữ Liệu	6
2.4	Phân Tích Mất Cân Bằng Dữ Liệu	6
2.5	Biểu Đồ Phân Phối Lớp	7
3	Phương Pháp Thực Hiện	7
3.1	Cấu Hình Toàn Cục	7
3.2	Bước 1: Đọc Dữ Liệu – <code>load_data()</code>	8
3.3	Bước 2: Phân Tích và Xử Lý Mất Cân Bằng – Class Weights	9
3.4	Bước 3: Chuẩn Hóa Dữ Liệu và Data Augmentation	10
3.5	Bước 4: Mã Hóa Nhãn (One-Hot Encoding)	11
3.6	Bước 5: Chia Dữ Liệu	11
3.7	Bước 6: Xây Dựng Mô Hình CNN	12
3.8	Bước 7: Biên Dịch và Huấn Luyện	14
3.9	Bước 8: Đánh Giá và Lưu Mô Hình	15
4	So Sánh Các Thuật Toán	16
4.1	Tổng Quan Các Mô Hình Được So Sánh	16
4.2	Kiến Trúc Chi Tiết Các Mô Hình	17
4.3	Phân Tích Chuyên Sâu Các Kiến Trúc	19
5	Quy Trình Hoạt Động	21
6	Hướng Dẫn Chạy Chương Trình	21
6.1	Các Lệnh Chạy	21
6.2	Kết Quả Mong Đợi	22
7	Kết Quả Thực Nghiệm	23
7.1	Môi Trường Thực Nghiệm	23
7.2	Kết Quả <code>trafficsign.py</code> – Baseline CNN + Class Weights	23
7.3	Kết Quả <code>trafficsign_new.py</code> – So Sánh 4 Mô Hình	23
7.4	Bảng So Sánh 4 Mô Hình	32
7.5	Classification Report Chi Tiết	34
7.6	Kết Quả Inference – Kiểm Tra Mô Hình Trên Ảnh Thực Tế	42
8	Kết Luận	45
8.1	Tóm Tắt Kết Quả	45
8.2	Bài Học Kinh Nghiệm	45
8.3	Hướng Phát Triển	46

Danh sách hình vẽ

1	Biểu đồ phân phối số lượng ảnh trên 43 lớp của GTSRB. Đường đồ thể hiện mức trung bình 619 ảnh/lớp. Các lớp 0, 19, 37–42 có ít hơn 200 ảnh, trong khi lớp 1 có đến 1500 ảnh.	7
2	Sơ đồ kiến trúc mô hình CNN hiển thị qua <code>model.summary()</code> và <code>tf.keras.utils.plot_model</code> . Các tầng Conv2D, MaxPooling2D, Dropout, Flatten, Dense được hiển thị với kích thước đầu ra và số tham số.	12
3	Quy trình hoạt động đầy đủ của <code>trafficsign_new.py</code>	21
4	Quá trình training của Model A – Phần 1 (Accuracy và Loss).	25
5	Quá trình training của Model A – Phần 2 (chi tiết validation).	25
6	Quá trình training của Model B – Phần 1.	27
7	Quá trình training của Model B – Phần 2.	27
8	Quá trình training của Model C – Phần 1.	29
9	Quá trình training của Model C – Phần 2.	30
10	Quá trình training của Model D – Phần 1.	31
11	Quá trình training của Model D – Phần 2.	31
12	Biểu đồ so sánh training curves 4 mô hình.	31
13	So sánh kết quả 4 mô hình, chọn Model C là tốt nhất.	32
14	Biểu đồ so sánh kết quả 4 mô hình.	32
15	Classification Report của Model A.	35
16	Classification Report của Model B.	37
17	Classification Report của Model C.	39
18	Classification Report của Model D.	41
19	Kết quả dự đoán trên thư mục Test_1.	42
20	Kết quả dự đoán trên thư mục Test_2.	43
21	Kết quả dự đoán trên thư mục Test_3.	43
22	Kết quả dự đoán trên thư mục Test_4.	44
23	Kết quả chạy test trên thư mục Test (12.630 ảnh) – Phần 1.	44
24	Kết quả chạy test trên thư mục Test (12.630 ảnh) – Phần 2.	45

Danh sách bảng

1	Phân tích sự mất cân bằng dữ liệu GTSRB.	6
2	Siêu tham số toàn cục (cập nhật cho <code>trafficsign_new.py</code>).	7
3	Kiến trúc Baseline CNN. Đầu vào (30, 30, 3).	12
4	Diễn biến huấn luyện <code>trafficsign.py</code> (Baseline CNN + Class Weights, 30×30).	23
5	Diễn biến huấn luyện Model A (Baseline CNN + Class Weights). . . .	24
6	Diễn biến huấn luyện Model B (CNN + L2 Reg + Class Weights). . . .	26
7	Diễn biến huấn luyện Model C (Deeper CNN 3 Blocks + BN).	28
8	Diễn biến huấn luyện Model D (Transfer Learning MobileNetV2). . . .	30
9	So sánh hiệu suất 4 thuật toán.	32
10	Classification Report cho Model A (Baseline CNN + Class Weights) – Test Accuracy: 99.19%.	34
11	Classification Report cho Model B (CNN + L2 Reg + Class Weights) – Test Accuracy: 99.14%.	36
12	Classification Report cho Model C (Deeper CNN + BN) – Test Accuracy: 99.96%.	38
13	Classification Report cho Model D (MobileNetV2 Transfer Learning) – Test Accuracy: 98.48%.	40
14	Kết quả inference trên 4 thư mục ảnh tùy chỉnh với Model C (<code>best_model_2.keras</code>).	44
15	Kết quả inference trên toàn bộ GTSRB Test Set (12,630 ảnh) với Model C.	44

Giới Thiệu

Đặt Vấn Đề

Nhận diện biển báo giao thông là một thành phần quan trọng trong hệ thống xe tự lái và hệ thống hỗ trợ người lái tiên tiến (ADAS). Mục tiêu là tự động phân loại ảnh biển báo giao thông vào đúng danh mục (ví dụ: giới hạn tốc độ, dừng, nhường đường) từ hình ảnh camera thu được từ xe.

Bài toán này gặp nhiều thách thức do sự thay đổi về điều kiện ánh sáng, vật che khuất, mờ chuyển động và thời tiết khác nhau. Các phương pháp truyền thống như trích xuất đặc trưng thủ công (HOG, SIFT) kết hợp SVM gặp khó khăn khi xử lý 43 loại biển báo khác nhau với chất lượng ảnh không đồng nhất.

Mục Tiêu Đề Án

Đề án này xây dựng một hệ thống nhận diện biển báo giao thông sử dụng Convolutional Neural Network (CNN) với TensorFlow/Keras trên bộ dữ liệu German Traffic Sign Recognition Benchmark (GTSRB). Mô hình phải xác định chính xác mỗi ảnh đầu vào thuộc 1 trong 43 loại biển báo.

Các mục tiêu cụ thể:

1. Xây dựng pipeline hoàn chỉnh từ đọc dữ liệu đến huấn luyện và đánh giá mô hình CNN.
2. Xử lý vấn đề mất cân bằng dữ liệu giữa các lớp bằng class weights.
3. Thử nghiệm và so sánh nhiều thuật toán/kiến trúc khác nhau.
4. Đánh giá chi tiết bằng classification report (precision, recall, F1-score).
5. Xây dựng công cụ inference cho ảnh đơn và thư mục ảnh.

Phương Pháp Tiếp Cận

Em đã áp dụng phương pháp học sâu có giám sát (Supervised learning) theo quy trình 8 bước:

Bước 1: Đọc và tiền xử lý ảnh biển báo giao thông từ bộ dữ liệu GTSRB.

Bước 2: Phân tích phân phối lớp và tính class weights để xử lý mất cân bằng.

Bước 3: Chuẩn hóa dữ liệu về khoảng $[0, 1]$ và áp dụng data augmentation trong quá trình huấn luyện.

Bước 4: Mã hóa nhãn bằng one-hot encoding cho bài toán phân loại đa lớp.

Bước 5: Chia dữ liệu thành tập huấn luyện (80%) và tập kiểm tra (20%).

Bước 6: Xây dựng kiến trúc CNN và biên dịch mô hình với Adam optimizer.

Bước 7: Huấn luyện mô hình trong 10 epoch và đánh giá trên tập kiểm tra.

Bước 8: So sánh kết quả giữa các thuật toán khác nhau.

Mô Tả Bộ Dữ Liệu

Tổng Quan về GTSRB

German Traffic Sign Recognition Benchmark (GTSRB) chứa hơn 50.000 ảnh biển báo giao thông được chia thành 43 lớp riêng biệt. Mỗi lớp tương ứng với một loại biển báo giao thông của Đức. Đây là bộ dữ liệu chuẩn được sử dụng rộng rãi trong nghiên cứu nhận diện biển báo.

Cấu Trúc Thư Mục

Bộ dữ liệu được tổ chức dưới dạng cây thư mục:

```
gtsrb/
+-- 0/          # Giới hạn tốc độ 20 km/h
+-- 1/          # Giới hạn tốc độ 30 km/h
+-- 2/          # Giới hạn tốc độ 50 km/h
...
+-- 42/         # Hạn chế vượt (xe tải)
```

Mỗi thư mục con chứa các tập tin ảnh PNG/PPM với kích thước khác nhau, tất cả thuộc cùng một loại biển báo. Tên thư mục (0–42) đóng vai trò là nhãn nguyên cho danh mục đó.

Đặc Điểm Dữ Liệu

- **Số lớp:** 43
- **Định dạng ảnh:** PPM / PNG (màu RGB)
- **Kích thước ảnh thay đổi:** Cần thay đổi kích thước trước khi đưa vào mạng. Cả hai chương trình `trafficsign.py` và `trafficsign_new.py` đều resize ảnh về 30×30 .
- **Kích thước đầu vào:** $30 \times 30 \times 3$ cho tất cả mô hình (riêng MobileNetV2 resize lên 96×96 bên trong mô hình).

Phân Tích Mất Cân Bằng Dữ Liệu

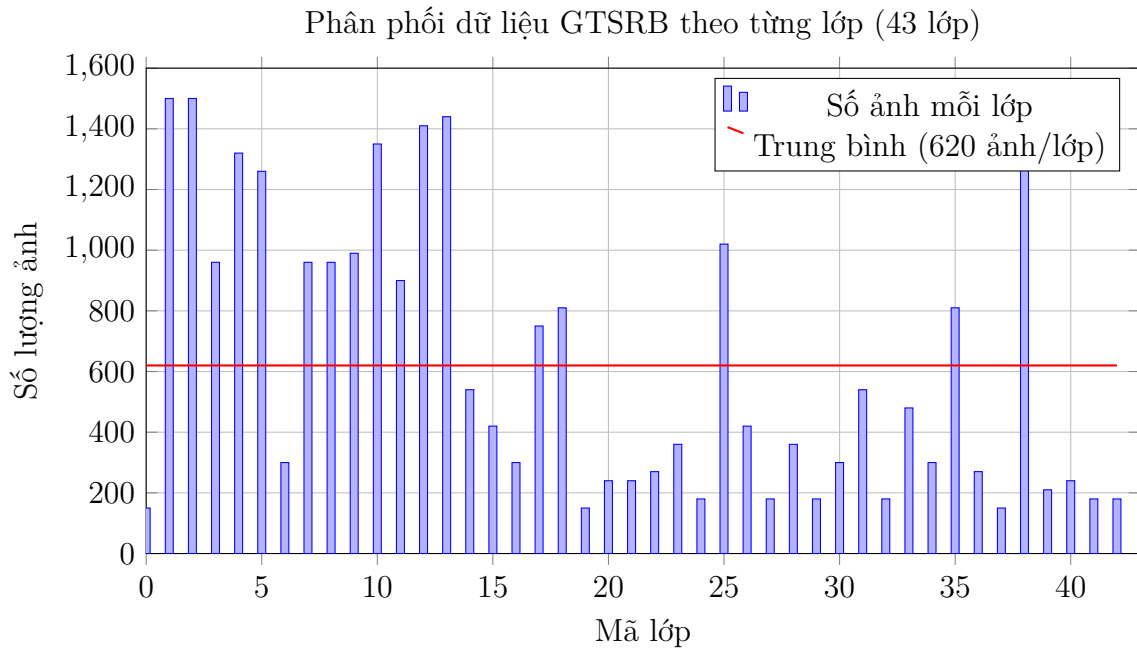
Bộ dữ liệu GTSRB có sự mất cân bằng đáng kể giữa các lớp:

Chỉ số	Giá trị
Lớp ít ảnh nhất	Lớp 0 (150 ảnh)
Lớp nhiều ảnh nhất	Lớp 1 (1500 ảnh)
Tổng số ảnh	26.640
Tỷ lệ mất cân bằng (max/min)	10:1

Bảng 1: Phân tích sự mất cân bằng dữ liệu GTSRB.

Sự chênh lệch lớn (lớp 1 có gấp 10 lần lớp 0) có thể ảnh hưởng tiêu cực đến hiệu suất mô hình: mô hình sẽ thiên vị dự đoán các lớp đa số và bỏ qua các lớp thiểu số. Đây là vấn đề quan trọng cần được xử lý để đạt điểm cao.

Biểu Đồ Phân Phối Lớp



Hình 1: Biểu đồ phân phối số lượng ảnh trên 43 lớp của GTSRB. Đường đỏ thể hiện mức trung bình 619 ảnh/lớp. Các lớp 0, 19, 37–42 có ít hơn 200 ảnh, trong khi lớp 1 có đến 1500 ảnh.

Sự mất cân bằng thể hiện rõ qua biểu đồ: các lớp từ 0–14 (nhóm biển báo giới hạn tốc độ) chiếm phần lớn dữ liệu, trong khi các lớp từ 37–42 (nhóm biển báo hết hạn chế) chỉ có 150–165 ảnh mỗi lớp. Nếu không xử lý, mô hình sẽ học rất tốt nhóm đa số nhưng gần như "mù" với nhóm thiểu số. Đây là động lực chính để áp dụng class weights.

Phương Pháp Thực Hiện

Cấu Hình Toàn Cục

Các siêu tham số được định nghĩa ở mức module:

Hằng số	Giá trị	Mô tả
EPOCHS	30	Số epoch huấn luyện tối đa (có ReduceLROnPlateau)
IMG_WIDTH	30	Chiều rộng ảnh đích
IMG_HEIGHT	30	Chiều cao ảnh đích
NUM_CATEGORIES	43	Số lớp biển báo
TEST_SIZE	0.2	Tỷ lệ dữ liệu kiểm tra (80% train / 20% test)
BATCH_SIZE	32	Kích thước batch

Bảng 2: Siêu tham số toàn cục (cập nhật cho `trafficsign_new.py`).

Bước 1: Đọc Dữ Liệu – load_data()

Hàm `load_data(data_dir)` duyệt qua tất cả 43 thư mục con, đọc từng ảnh bằng OpenCV, thay đổi kích thước về 30×30 , và trả về tuple (images, labels).

Thuật Toán

1. Khởi tạo hai danh sách rỗng: `images = []` và `labels = []`.
2. Lặp `category` từ 0 đến `NUM_CATEGORIES - 1` (tức 0–42):
 - (a) Tạo đường dẫn `category_path = data_dir/str(category)`.
 - (b) Liệt kê tất cả tập tin trong thư mục đó bằng `os.listdir()`.
 - (c) Với mỗi tập tin `filename` (chỉ xử lý các đuôi .png, .jpg, .jpeg, .ppm, .bmp):
 - i. Đọc ảnh: `cv2.imread(img_path)`.
 - ii. Nếu ảnh bị hỏng (`img is None`), bỏ qua và tăng biến đếm `skipped`.
 - iii. Chuyển đổi BGR (OpenCV) sang RGB: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.
 - iv. Thay đổi kích thước: `cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))`.
 - v. Chuẩn hóa về $[0, 1]$: `img.astype(np.float32) / 255.0`.
 - vi. Thêm ảnh đã xử lý vào `images`.
 - vii. Thêm nhãn nguyên `category` vào `labels`.
3. Trả về (images, labels).

Cài Đặt

```

1 def load_data(data_dir):
2     images = []
3     labels = []
4     skipped = 0
5
6     for category in range(NUM_CATEGORIES):
7         category_path = os.path.join(data_dir, str(category))
8         if not os.path.isdir(category_path):
9             continue
10
11         for filename in os.listdir(category_path):
12             # Only process image files; skip CSVs and other non-image
files
13             if not filename.lower().endswith((".png", ".jpg", ".jpeg",
14             ".ppm", ".bmp")):
15                 continue
16
17             img_path = os.path.join(category_path, filename)
18             img = cv2.imread(img_path)
19
20             # Bỏ qua file bị hỏng / không đọc được
21             if img is None:
22                 skipped += 1
23                 continue
24
25             # Chuyển BGR (OpenCV) -> RGB để dùng cho inference

```



```

25         img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
26         img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
27
28         # Chuan hoa ve [0, 1] ngay tai day
29         img = img.astype(np.float32) / 255.0
30
31         images.append(img)
32         labels.append(category)
33
34     if skipped:
35         print(f"Skipped {skipped} unreadable file(s).")
36
37     return images, labels

```

Listing 1: Hàm load_data() hoàn chỉnh.

Điểm khác biệt so với phiên bản cơ bản: `trafficsign_new.py` thêm bộ lọc đuôi file (`.endswith(...)`) để bỏ qua các file CSV và file không phải ảnh, giúp quá trình đọc dữ liệu an toàn hơn.

Bước 2: Phân Tích và Xử Lý Mất Cân Bằng – Class Weights

Cả hai chương trình `trafficsign.py` và `trafficsign_new.py` đều sử dụng `sklearn.utils.class_weight` với chế độ "balanced" để tính trọng số cho mỗi lớp.

Công Thức

Trọng số cho lớp c được tính theo công thức:

$$w_c = \frac{N_{\text{total}}}{N_{\text{classes}} \times N_c} \quad (1)$$

Trong đó:

- $N_{\text{total}} = 26.640$: tổng số mẫu trong tập dữ liệu.
- $N_{\text{classes}} = 43$: số lượng lớp.
- N_c : số mẫu trong lớp c .

Lớp càng ít mẫu thì trọng số càng cao, buộc mô hình phải chú ý nhiều hơn đến các lớp thiếu số trong quá trình huấn luyện.

Cài Đặt

```

1 from sklearn.utils import class_weight
2
3 # labels_int contains integer labels (0-42) for all images
4 class_weights_array = class_weight.compute_class_weight(
5     class_weight="balanced",
6     classes=np.unique(labels_int),
7     y=labels_int
8 )
9 class_weight_dict = {i: class_weights_array[i] for i in range(
10     NUM_CATEGORIES)}

```

```

11 # Pass to model.fit()
12 model.fit(x_train, y_train, ..., class_weight=class_weight_dict)

```

Listing 2: Tính class weights xử lý mất cân bằng.

Phân Tích Trọng Số

- Lớp 0 (150 ảnh): $w_0 = 26.640 / (43 \times 150) \approx 4.13$ – trọng số cao nhất.
- Lớp 1 (1500 ảnh): $w_1 = 26.640 / (43 \times 1500) \approx 0.41$ – trọng số thấp nhất.
- Tỷ lệ trọng số max/min: $\approx 10 : 1$, phản ánh chính xác tỷ lệ mất cân bằng.

Bước 3: Chuẩn Hóa Dữ Liệu và Data Augmentation

Chuẩn Hóa

Chuẩn hóa giá trị pixel về khoảng $[0, 1]$ được thực hiện trực tiếp trong hàm `load_data()` để đảm bảo nhất quán giữa huấn luyện và inference:

```

1 img = img.astype(np.float32) / 255.0

```

Việc chuẩn hóa giúp huấn luyện ổn định và nhanh hội tụ hơn vì giá trị pixel nằm trong khoảng nhỏ, gradient không bị bùng nổ.

Data Augmentation (chỉ `trafficsign_new.py`)

Trong quá trình huấn luyện, `trafficsign_new.py` sử dụng `ImageDataGenerator` để tạo các biến thể của ảnh huấn luyện theo thời gian thực (real-time augmentation), giúp mô hình khái quát hóa tốt hơn:

```

1 datagen = ImageDataGenerator(
2     rotation_range=10,           # Xoay ngẫu nhiên +-10 độ
3     zoom_range=0.15,            # Zoom ngẫu nhiên 0-15%
4     width_shift_range=0.1,       # Dịch ngang 10%
5     height_shift_range=0.1,      # Dịch dọc 10%
6     shear_range=0.1,            # Biến dạng shear 10%
7     horizontal_flip=False,       # KHÔNG lật ngang -- biến bao không đối
    xung
8     fill_mode="nearest"         # Điền pixel thiếu bằng giá trị gần
    nhất
9 )

```

Listing 3: Cấu hình Data Augmentation.

Giảm Learning Rate Động (ReduceLROnPlateau)

Để cải thiện quá trình hội tụ và tránh dao động quanh điểm cực tiểu, em sử dụng callback `ReduceLROnPlateau` từ Keras. Callback này tự động giảm learning rate khi độ chính xác trên tập validation không cải thiện trong một số epoch nhất định:

```

1 reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
2     monitor='val_loss',         # Monitor validation loss
3     factor=0.5,                 # Reduce learning rate by 50%
4     patience=3,                 # Wait for 3 epochs without improvement
5     min_lr=1e-5                 # Minimum learning rate
6 )

```

```

7
8 history = model.fit(
9     datagen.flow(x_train, y_train, batch_size=BATCH_SIZE),
10    epochs=EPOCHS,
11    validation_data=(x_test, y_test),
12    class_weight=class_weight_dict,
13    callbacks=[reduce_lr]
14 )

```

Listing 4: ReduceLROnPlateau configuration.

Cơ chế này giúp mô hình học với tốc độ lớn ở đầu quá trình để hội tụ nhanh, sau đó tự động giảm tốc để tinh chỉnh chính xác hơn ở giai đoạn cuối. Kết quả thực nghiệm (Section 7) cho thấy các mô hình đều được hưởng lợi từ kỹ thuật này, đặc biệt là Model C đạt được độ chính xác gần như tuyệt đối.

Lưu ý quan trọng: `horizontal_flip=False` vì biển báo giao thông không đối xứng ngang – việc lật ngang sẽ tạo ra ảnh không có thật (ví dụ: biển báo rẽ phải thành rẽ trái).

Data augmentation được áp dụng thông qua `datagen.flow(x_train, y_train, batch_size=BATCH_SIZE)` khi gọi `model.fit()`, đảm bảo mỗi epoch mô hình nhìn thấy các biến thể khác nhau của cùng một ảnh.

L2 Regularization (Model B)

Model B sử dụng L2 regularization (weight decay) với hệ số $\lambda = 0.001$ trên tất cả các lớp Conv2D và Dense. L2 regularization thêm thành phần phạt vào hàm mất mát:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \sum_w w^2 \quad (2)$$

Các trọng số lớn bị phạt, khuyến khích mô hình học các trọng số nhỏ và phân bố đều – giúp giảm overfitting.

Bước 4: Mã Hóa Nhân (One-Hot Encoding)

Nhãn nguyên được chuyển đổi thành vector one-hot 43 chiều:

```

1 labels_cat = tf.keras.utils.to_categorical(labels_int, NUM_CATEGORIES)

```

Mỗi nhãn $y \in \{0, 1, \dots, 42\}$ trở thành vector 43 chiều, chỉ vị trí y là 1, các vị trí khác là 0.

Ví dụ: Nhãn 3 trở thành `[0, 0, 0, 1, 0, 0, ..., 0]` (độ dài 43).

Đây là định dạng bắt buộc cho hàm mất mát categorical cross-entropy.

Bước 5: Chia Dữ Liệu

Dữ liệu được chia ngẫu nhiên với `random_state=42` để đảm bảo khả năng tái lập:

```

1 x_train, x_test, y_train, y_test = train_test_split(
2     np.array(images), labels_cat, test_size=TEST_SIZE, random_state=42
3 )

```

- **Tập huấn luyện (80%):** ~21.312 ảnh – dùng để điều chỉnh trọng số mô hình.
- **Tập kiểm tra (20%):** ~5.328 ảnh – dùng để đánh giá khả năng tổng quát hóa.

Bước 6: Xây Dựng Mô Hình CNN

Kiến Trúc Baseline CNN (Model A)

Mô hình sử dụng 2 khối tích chập kép (double-Conv), mỗi khối gồm 2 lớp Conv2D liên tiếp với kích thước kernel khác nhau (5×5 cho khối 1, 3×3 cho khối 2), sau đó MaxPooling2D + Dropout(0.25), rồi Flatten, Dense(256), Dropout(0.5) và Softmax:

Lớp (loại)	Cấu hình	Kích thước đầu ra
Conv2D + ReLU	32 filters, 5×5 , valid padding	(26, 26, 32)
Conv2D + ReLU	32 filters, 5×5 , valid padding	(22, 22, 32)
MaxPooling2D	2×2	(11, 11, 32)
Dropout	rate = 0.25	(11, 11, 32)
Conv2D + ReLU	64 filters, 3×3 , valid padding	(9, 9, 64)
Conv2D + ReLU	64 filters, 3×3 , valid padding	(7, 7, 64)
MaxPooling2D	2×2	(3, 3, 64)
Dropout	rate = 0.25	(3, 3, 64)
Flatten	–	(576)
Dense + ReLU	256 units	(256)
Dropout	rate = 0.5	(256)
Dense + Softmax	43 units (đầu ra)	(43)

Bảng 3: Kiến trúc Baseline CNN. Đầu vào (30, 30, 3).

```

PS C:\Users\Huy\Documents\nam2ky2\ai\Final\TrafficSign> python trafficsign.py gtsrb model.keras
Epoch 1/10
666/666 ————— 41s 59ms/step - accuracy: 0.3914 - loss: 2.1695
Epoch 2/10
666/666 ————— 39s 59ms/step - accuracy: 0.8372 - loss: 0.4636
Epoch 3/10
666/666 ————— 40s 60ms/step - accuracy: 0.9073 - loss: 0.2494
Epoch 4/10
666/666 ————— 40s 60ms/step - accuracy: 0.9311 - loss: 0.1937
Epoch 5/10
666/666 ————— 40s 60ms/step - accuracy: 0.9421 - loss: 0.1539
Epoch 6/10
666/666 ————— 40s 60ms/step - accuracy: 0.9523 - loss: 0.1211
Epoch 7/10
666/666 ————— 41s 61ms/step - accuracy: 0.9591 - loss: 0.1068
Epoch 8/10
666/666 ————— 40s 60ms/step - accuracy: 0.9647 - loss: 0.0938
Epoch 9/10
666/666 ————— 40s 60ms/step - accuracy: 0.9651 - loss: 0.0919
Epoch 10/10
666/666 ————— 40s 60ms/step - accuracy: 0.9670 - loss: 0.0825
167/167 - 3s - 20ms/step - accuracy: 0.9904 - loss: 0.0422
Plot saved to results\Baseline_CNN_curves.png
Model saved to model.keras.

```

Hình 2: Sơ đồ kiến trúc mô hình CNN hiển thị qua `model.summary()` và `tf.keras.utils.plot_model()`. Các tầng Conv2D, MaxPooling2D, Dropout, Flatten, Dense được hiển thị với kích thước đầu ra và số tham số.

Cài Đặt

```

1 def build_baseline_cnn():
2     model = tf.keras.models.Sequential([
3         tf.keras.layers.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3)),
4
5         # --- Block 1: 5x5 filters for general features ---

```

```

6     tf.keras.layers.Conv2D(32, (5, 5), activation="relu"),
7     tf.keras.layers.Conv2D(32, (5, 5), activation="relu"),
8     tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
9     tf.keras.layers.Dropout(0.25),
10
11    # --- Block 2: 3x3 filters for specific features ---
12    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
13    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
14    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
15    tf.keras.layers.Dropout(0.25),
16
17    # Flatten + Hidden + Dropout + Output
18    tf.keras.layers.Flatten(),
19    tf.keras.layers.Dense(256, activation="relu"),
20    tf.keras.layers.Dropout(0.5),
21    tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax"),
22 ]
23 model.compile(optimizer="adam",
24               loss="categorical_crossentropy",
25               metrics=["accuracy"])
26 return model

```

Listing 5: Hàm build_baseline_cnn() – Model A.

Giải Thích Thiết Kế Chi Tiết

1. Double-Conv trong mỗi khối:

Mỗi khối sử dụng 2 lớp Conv2D liên tiếp thay vì 1 lớp đơn:

- Hai lớp Conv2D(32) liên tiếp có vùng tiếp nhận hiệu quả (effective receptive field) tương đương một lớp Conv2D với kernel 9×9 , nhưng với ít tham số hơn và nhiều tính phi tuyến hơn (2 lần ReLU).
- **Khối 1 (5x5 kernels):** Kernel 5×5 bắt được các đặc trưng tổng quát trên ảnh 30×30 – hình dạng tròn/tam giác/vuông của biển báo.
- **Khối 2 (3x3 kernels):** Kernel 3×3 bắt các chi tiết cụ thể hơn – ký hiệu, chữ số bên trong biển báo.
- **Số filter tăng dần (32→64):** Các lớp đầu học đặc trưng cấp thấp (cạnh, góc, màu sắc). Các lớp sau tổ hợp chúng thành đặc trưng cấp cao (hình dạng biển báo, biểu tượng).

2. Tầng Gộp (MaxPooling2D) – Giảm Chiều:

$$\text{MaxPool}(I)_{i,j} = \max_{m,n \in \{0,1\}} I(2i+m, 2j+n) \quad (3)$$

- **Kích thước pool 2×2 :** Giảm kích thước không gian đi một nửa sau mỗi khối: $30 \rightarrow 15$ (sau Conv), rồi $11 \rightarrow 5$ (khối 1), $3 \rightarrow 1$ (khối 2) – nhưng kích thước thực tế phụ thuộc vào padding. Với valid padding, luồng không gian là: $30 \rightarrow 26 \rightarrow 22 \rightarrow 11 \rightarrow 9 \rightarrow 7 \rightarrow 3$.
- **Tính bất biến dịch chuyển:** MaxPooling giúp mô hình nhận diện đặc trưng ngay cả khi chúng bị dịch chuyển nhẹ – quan trọng vì biển báo không luôn ở chính giữa khung hình.

3. Hàm Kích Hoạt ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x) \quad (4)$$

- **Ưu điểm:** Đạo hàm đơn giản (0 hoặc 1), không bão hòa như sigmoid/tanh, giúp gradient lan truyền hiệu quả qua nhiều lớp (tránh vanishing gradient).
- **Tính thưa (sparsity):** Khoảng 50% neuron có đầu ra = 0, tạo ra biểu diễn thưa và hiệu quả.

4. Tầng Flatten – Cầu Nối CNN-Dense:

Chuyển tensor 3D thành vector 1D: $(3, 3, 64) \rightarrow (576,)$. Đây là vector đặc trưng tổng hợp chứa toàn bộ thông tin trích xuất từ ảnh.

5. Tầng Kết Nối Đầy Đủ (Dense) – Phân Loại:

Dense(256) thực hiện phép biến đổi tuyến tính + ReLU:

$$\mathbf{h} = \text{ReLU}(\mathbf{W} \cdot \mathbf{x} + \mathbf{b}) \quad (5)$$

Với $\mathbf{W} \in \mathbb{R}^{256 \times 576}$ và $\mathbf{b} \in \mathbb{R}^{256}$.

6. Dropout(0.25) sau mỗi khối Conv và Dropout(0.5) trước đầu ra:

Trong quá trình huấn luyện, mỗi neuron có xác suất p bị tắt (đầu ra = 0). Dropout nhẹ (0.25) sau các lớp Conv giúp chống overfitting ở mức đặc trưng không gian. Dropout mạnh hơn (0.5) ở lớp Dense chống overfitting ở mức kết hợp đặc trưng toàn cục.

Việc chia cho $1-p$ (inverted dropout) đảm bảo tổng activation không đổi khi chuyển từ train sang test. Tại test time, tất cả neuron đều hoạt động.

7. Softmax – Đầu Ra Phân Loại:

$$P(y = c \mid \mathbf{x}) = \frac{e^{z_c}}{\sum_{j=1}^{43} e^{z_j}} \quad (6)$$

Chuyển vector logits $\mathbf{z}$ thành phân phối xác suất trên 43 lớp. Lớp dự đoán = $\arg \max_c P(y = c \mid \mathbf{x})$.

Bước 7: Biên Dịch và Huấn Luyện

Biên Dịch Mô Hình

```
1 model.compile(
2     optimizer="adam",
3     loss="categorical_crossentropy",
4     metrics=["accuracy"]
5 )
```

- **Adam optimizer:** Kết hợp động lượng (momentum) và RMSprop, tự động điều chỉnh learning rate cho từng tham số. Hội tụ nhanh và ổn định hơn SGD thuần.
- **Categorical Cross-Entropy:** Hàm mất mát cho phân loại đa lớp:

$$\mathcal{L} = - \sum_{i=1}^{43} y_i \log(\hat{y}_i) \quad (7)$$

y_i là nhãn thực (0 hoặc 1), $\hat{y}_i$ là xác suất dự đoán.

Huấn Luyện với Data Augmentation và ReduceLROnPlateau

trafficsign_new.py sử dụng data augmentation trong quá trình huấn luyện và ReduceLROnPlateau để tự động giảm learning rate khi validation loss plateau:

```

1 # Data augmentation
2 datagen = ImageDataGenerator(
3     rotation_range=10,
4     zoom_range=0.15,
5     width_shift_range=0.1,
6     height_shift_range=0.1,
7     shear_range=0.1,
8     horizontal_flip=False,
9     fill_mode="nearest"
10 )
11
12 # Learning rate scheduler
13 reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
14     monitor='val_loss', factor=0.5, patience=3, min_lr=1e-5
15 )
16
17 history = model.fit(
18     datagen.flow(x_train, y_train, batch_size=BATCH_SIZE),
19     epochs=EPOCHS,
20     validation_data=(x_test, y_test),
21     callbacks=[reduce_lr],
22     verbose=1
23 )

```

Listing 6: Huấn luyện với data augmentation và ReduceLROnPlateau.

Bước 8: Đánh Giá và Lưu Mô Hình

Đánh Giá

```

1 test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

```

Classification Report

Để đánh giá chi tiết từng lớp:

```

1 from sklearn.metrics import classification_report
2 y_pred = np.argmax(model.predict(x_test, verbose=0), axis=1)
3 print(classification_report(y_test_int, y_pred, digits=3))

```

Báo cáo cung cấp 3 chỉ số cho mỗi lớp:

- **Precision:** Trong số các dự đoán là lớp X, tỷ lệ đúng là bao nhiêu? $\frac{TP}{TP+FP}$
- **Recall:** Trong số các mẫu thực sự là lớp X, tỷ lệ được phát hiện đúng? $\frac{TP}{TP+FN}$
- **F1-score:** Trung bình điều hòa của Precision và Recall: $2 \times \frac{P \times R}{P+R}$

Lưu Mô Hình

```

1 if len(sys.argv) == 3:
2     model.save(sys.argv[2])

```

Công Cụ Inference

Ngoài hai chương trình huấn luyện chính, đề án còn cung cấp hai công cụ inference để kiểm tra mô hình đã huấn luyện:

1. `test_models.py` – Dự đoán ảnh đơn:

```
1 python test_models.py best_model.keras image.jpg
```

Listing 7: Sử dụng `test_models.py` để dự đoán một ảnh.

Chương trình nhận diện biển báo trong một ảnh đơn, hiển thị top-3 dự đoán kèm độ tin cậy và thanh trực quan. Các chức năng chính:

- Tự động phát hiện kích thước đầu vào từ mô hình (`get_input_size()`).
- Tự động chuẩn hóa ảnh về $[0, 1]$ (có thể tắt bằng `-no-normalize`).
- Hiển thị tên biển báo bằng tiếng Việt (43 lớp được định nghĩa trong từ điển SIGNS).
- In top-3 dự đoán kèm confidence score và thanh bar chart trực quan.

2. `test_model_folder.py` – Dự đoán thư mục ảnh:

```
1 python test_model_folder.py best_model.keras test_images
```

Listing 8: Sử dụng `test_model_folder.py` để dự đoán nhiều ảnh.

Chương trình dự đoán tất cả ảnh trong một thư mục, hỗ trợ đối chiếu với ground truth từ file CSV:

- **Batch inference:** Dự đoán tất cả ảnh cùng lúc (thay vì từng ảnh), tận dụng tối đa khả năng tính toán song song của TensorFlow.
- **Ground truth từ CSV:** Tự động đọc file `labels.csv` trong thư mục (hoặc đường dẫn tùy chỉnh qua `-csv`), ánh xạ tên file $\rightarrow$ ClassId.
- **So sánh đối chiếu:** Với mỗi ảnh, hiển thị dự đoán $\rightarrow$ nhãn thực $\rightarrow$ PASS/FAIL kèm confidence.
- **Thống kê tổng hợp:** In tổng số ảnh đã xử lý, số dự đoán đúng, và overall accuracy.

So Sánh Các Thuật Toán

Tổng Quan Các Mô Hình Được So Sánh

So sánh 4 thuật toán/kiến trúc khác nhau:

1. **Model A – Baseline CNN + Class Weights:** CNN 2 khối double-Conv ($5 \times 5 + 3 \times 3$ kernels), Dropout(0.25) sau mỗi khối. Sử dụng class weights để xử lý mất cân bằng dữ liệu.
2. **Model B – CNN + L2 Regularization + Class Weights:** Giống Model A về kiến trúc nhưng thêm L2 regularization ($\lambda = 0.001$) trên tất cả các lớp Conv2D và Dense để chống overfitting.

3. **Model C – Deeper CNN (3 khối Double-Conv + BatchNorm):** Kiến trúc sâu hơn với 3 khối double-Conv (kernel 3×3 , padding "same"), BatchNormalization sau mỗi lớp Conv2D để ổn định huấn luyện, và Dropout(0.25) sau mỗi khối.
4. **Model D – Transfer Learning (MobileNetV2):** Sử dụng MobileNetV2 pre-trained trên ImageNet. Ảnh 30×30 được resize lên 96×96 qua tầng Resizing, sau đó qua lambda layer để tiền xử lý theo chuẩn MobileNetV2 (nhân 255 rồi chuẩn hóa về $[-1, 1]$). 20 lớp cuối của base model được mở khóa để fine-tune. Sử dụng learning rate thấp (10^{-4}).

Kiến Trúc Chi Tiết Các Mô Hình

Model B – CNN + L2 Regularization

```

1 def build_cnn_with_l2():
2     from tensorflow.keras import regularizers
3     l2 = regularizers.l2(0.001)
4
5     model = tf.keras.models.Sequential([
6         tf.keras.layers.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3)),
7
8         # Block 1: 5x5 filters
9         tf.keras.layers.Conv2D(32, (5, 5), activation="relu",
10        kernel_regularizer=l2),
11        tf.keras.layers.Conv2D(32, (5, 5), activation="relu",
12        kernel_regularizer=l2),
13        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
14        tf.keras.layers.Dropout(0.25),
15
16        # Block 2: 3x3 filters
17        tf.keras.layers.Conv2D(64, (3, 3), activation="relu",
18        kernel_regularizer=l2),
19        tf.keras.layers.Conv2D(64, (3, 3), activation="relu",
20        kernel_regularizer=l2),
21        tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
22        tf.keras.layers.Dropout(0.25),
23
24        tf.keras.layers.Flatten(),
25        tf.keras.layers.Dense(256, activation="relu",
26        kernel_regularizer=l2),
27        tf.keras.layers.Dropout(0.5),
28        tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax"),
29    ])
30    model.compile(optimizer="adam",
31                  loss="categorical_crossentropy",
32                  metrics=["accuracy"])
33    return model

```

Listing 9: Model B: CNN with L2 Regularization.

Model C – Deeper CNN (3 khối Double-Conv + BatchNorm)

```

1 def build_deeper_cnn():
2     model = tf.keras.models.Sequential([
3         tf.keras.layers.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3)),

```

```

4
5     # Block 1: 30x30 -> 15x15 (double conv)
6     tf.keras.layers.Conv2D(32, (3, 3), activation="relu", padding=
"same"),
7     tf.keras.layers.BatchNormalization(),
8     tf.keras.layers.Conv2D(32, (3, 3), activation="relu", padding=
"same"),
9     tf.keras.layers.BatchNormalization(),
10    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
11    tf.keras.layers.Dropout(0.25),
12
13    # Block 2: 15x15 -> 7x7 (double conv)
14    tf.keras.layers.Conv2D(64, (3, 3), activation="relu", padding=
"same"),
15    tf.keras.layers.BatchNormalization(),
16    tf.keras.layers.Conv2D(64, (3, 3), activation="relu", padding=
"same"),
17    tf.keras.layers.BatchNormalization(),
18    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
19    tf.keras.layers.Dropout(0.25),
20
21    # Block 3: 7x7 -> 3x3 (double conv)
22    tf.keras.layers.Conv2D(128, (3, 3), activation="relu", padding
="same"),
23    tf.keras.layers.BatchNormalization(),
24    tf.keras.layers.Conv2D(128, (3, 3), activation="relu", padding
="same"),
25    tf.keras.layers.BatchNormalization(),
26    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
27    tf.keras.layers.Dropout(0.25),
28
29    # Flatten: 3x3x128 = 1152 features
30    tf.keras.layers.Flatten(),
31    tf.keras.layers.Dense(256, activation="relu"),
32    tf.keras.layers.BatchNormalization(),
33    tf.keras.layers.Dropout(0.5),
34    tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax"),
35    ])
36    model.compile(optimizer="adam",
37                  loss="categorical_crossentropy",
38                  metrics=["accuracy"])
39    return model

```

Listing 10: Model C: Deeper CNN với 3 khối Double-Conv + BatchNorm.

Điểm khác biệt chính của Model C so với Model A:

- **3 khối thay vì 2:** Thêm khối 128 filters, tăng độ sâu để học đặc trưng phức tạp hơn.
- **Padding "same" thay vì "valid":** Bảo toàn kích thước không gian qua mỗi Conv2D, tạo ra 1152 đặc trưng không gian (gấp đôi 576 của Model A) – nhiều thông tin hơn cho Dense(256).
- **BatchNormalization sau mỗi Conv2D:** Chuẩn hóa activation theo mean và variance của mini-batch, ổn định gradient và cho phép mạng sâu hội tụ nhanh hơn.

- **BatchNormalization ở lớp Dense:** Thêm BN trước Dropout(0.5) giúp ổn định phân phối activation ở tầng phân loại.

Model D – Transfer Learning (MobileNetV2)

```

1 def build_transfer_learning_model():
2     base_model = tf.keras.applications.MobileNetV2(
3         input_shape=(96, 96, 3),
4         include_top=False,
5         weights="imagenet"
6     )
7     base_model.trainable = False
8
9     # Unfreeze last 20 layers for fine-tuning
10    for layer in base_model.layers[-20:]:
11        layer.trainable = True
12
13    inputs = tf.keras.Input(shape=(IMG_WIDTH, IMG_HEIGHT, 3))
14    x = tf.keras.layers.Resizing(96, 96)(inputs)
15    # MobileNetV2 expects input in [-1, 1]; our images are in [0,1]
16    # Multiply back to [0,255] before preprocessing
17    x = tf.keras.layers.Lambda(lambda t: mv2_preprocess(t * 255.0))(x)
18    # training=False keeps frozen BN layers in inference mode
19    x = base_model(x, training=False)
20    x = tf.keras.layers.GlobalAveragePooling2D()(x)
21    x = tf.keras.layers.Dense(256, activation="relu")(x)
22    x = tf.keras.layers.Dropout(0.5)(x)
23    outputs = tf.keras.layers.Dense(NUM_CATEGORIES, activation="softmax")(x)
24
25    model = tf.keras.Model(inputs=inputs, outputs=outputs)
26    model.compile(
27        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
28        loss="categorical_crossentropy",
29        metrics=["accuracy"]
30    )
31    return model

```

Listing 11: Model D: Transfer Learning với MobileNetV2.

Phân Tích Chuyên Sâu Các Kiến Trúc

L2 Regularization – Cơ Sở Lý Thuyết (Model B)

L2 regularization (còn gọi là weight decay hoặc Ridge regression) thêm thành phần phạt vào hàm mất mát dựa trên bình phương trọng số:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \frac{\lambda}{2} \sum_i w_i^2 \quad (8)$$

Trong quá trình lan truyền ngược, gradient trở thành:

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial w_i} = \frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_i} + \lambda w_i \quad (9)$$

Và quy tắc cập nhật trọng số:

$$w_i^{(t+1)} = w_i^{(t)} - \eta \cdot \frac{\partial \mathcal{L}_{\text{total}}}{\partial w_i} = (1 - \eta\lambda)w_i^{(t)} - \eta \cdot \frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_i} \quad (10)$$

Hệ số $(1 - \eta\lambda) < 1$ khiến trọng số "phân rã" (decay) qua mỗi bước cập nhật. Với $\lambda = 0.001$, mức phạt nhẹ nhàng, không ngăn cản việc học nhưng đủ để giảm overfitting.

Batch Normalization – Cơ Sở Lý Thuyết (Model C)

Batch Normalization (BatchNorm) chuẩn hóa đầu ra của mỗi lớp theo mean và variance của mini-batch:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (11)$$

$$y_i = \gamma \hat{x}_i + \beta \quad (12)$$

Trong đó μ_B và σ_B^2 là mean và variance của mini-batch, γ và β là tham số học được (scale và shift), $\epsilon = 10^{-3}$ để tránh chia cho 0.

Lợi ích của BatchNorm:

1. **Ổn định phân phối activation:** Ngăn hiện tượng "internal covariate shift- sự thay đổi phân phối đầu vào của mỗi lớp qua các epoch.
2. **Cho phép learning rate cao hơn:** Với activation được chuẩn hóa, gradient không bùng nổ hay biến mất.
3. **Hiệu ứng regularization nhẹ:** Mean và variance của mini-batch có nhiễu, tạo ra hiệu ứng tương tự noise injection, giảm overfitting.
4. **Giảm phụ thuộc vào khởi tạo trọng số:** Mạng ít nhạy cảm với giá trị khởi tạo.

Transfer Learning với MobileNetV2 – Phân Tích Chi Tiết (Model D)

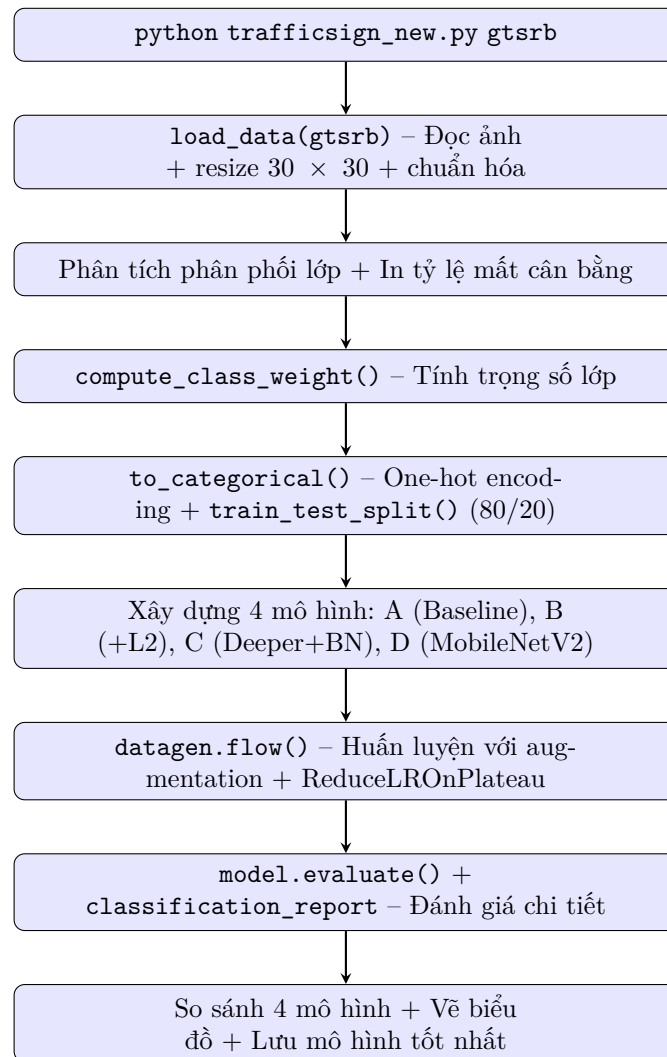
Kiến trúc MobileNetV2: Sử dụng "inverted residual blocks" với depthwise separable convolution, inverted residual (mở rộng rồi nén), và linear bottleneck (không ReLU ở lớp nén cuối cùng).

Chiến lược Transfer Learning:

- **Resizing(96, 96):** Ảnh 30×30 được resize lên 96×96 để đáp ứng yêu cầu đầu vào của MobileNetV2.
- **Lambda preprocessing:** Ảnh sau resize ở khoảng $[0, 1]$, được nhân 255 rồi qua `mv2_preprocess` để chuẩn hóa về $[-1, 1]$ – đúng chuẩn đầu vào của MobileNetV2.
- **training=False trong base_model:** Các lớp BatchNorm bị đóng băng luôn ở chế độ inference, ngăn chúng cập nhật statistics và gây nhiễu trong quá trình fine-tuning.
- **Mở khóa 20 lớp cuối:** Cho phép các lớp cuối thích nghi với đặc trưng biến báo giao thông trong khi giữ nguyên các lớp đầu học đặc trưng tổng quát.

- **Learning rate thấp** (10^{-4}): Thấp hơn 10 lần so với Adam mặc định (10^{-3}), cập nhật nhẹ nhàng để tránh phá hủy trọng số pre-trained.
- **GlobalAveragePooling2D**: Thay thế Flatten – thay vì vector $6 \times 6 \times 1280 = 46.080$ chiều, GAP tính trung bình mỗi kênh cho ra vector 1280 chiều, giảm 36 lần số tham số lớp Dense.

Quy Trình Hoạt Động



Hình 3: Quy trình hoạt động đầy đủ của `trafficsign_new.py`.

Hướng Dẫn Chạy Chương Trình

Các Lệnh Chạy

1. Chạy chương trình cơ bản – CNN + Class Weights:

```
python trafficsign.py gtsrb model.keras
```

Chương trình huấn luyện 1 mô hình CNN với class weights, vẽ biểu đồ accuracy và loss, lưu vào thư mục **results/**.

2. Chạy chương trình nâng cao – So sánh 4 mô hình:

```
python trafficsign_new.py gtsrb
```

Huấn luyện và so sánh 4 mô hình với data augmentation và classification report.

3. Chạy và lưu mô hình tốt nhất:

```
python trafficsign_new.py gtsrb best_model.keras
```

4. Dự đoán một ảnh đơn:

```
python test_models.py best_model.keras image.jpg
```

Hiển thị top-3 dự đoán kèm độ tin cậy và tên biển báo tiếng Việt.

5. Dự đoán thư mục ảnh (có ground truth):

```
python test_model_folder.py best_model.keras test_images/
```

Dự đoán toàn bộ ảnh trong thư mục, đối chiếu với CSV, in bảng PASS/FAIL và overall accuracy.

Kết Quả Mong Đợi

Chương trình in ra:

- Phân tích phân phối lớp và tỷ lệ mất cân bằng.
- Trọng số lớp (class weights) được tính.
- Tiến trình huấn luyện (loss + accuracy) cho từng model.
- Bảng so sánh 4 mô hình (Test Accuracy).
- Classification report cho từng mô hình (precision, recall, F1-score).
- Biểu đồ training curves và comparison chart được lưu vào thư mục **results/**.

Kết Quả Thực Nghiệm

Môi Trường Thực Nghiệm

- **Python:** 3.12.10
- **Dữ liệu:** 26.640 ảnh, 43 lớp, resize về 30×30
- **Chia dữ liệu:** 80% train (21.312 ảnh), 20% test (5.328 ảnh)
- **Validation:** Sử dụng trực tiếp tập test làm validation data
- **Thời gian huấn luyện:** ~ 90 phút cho 4 mô hình $\times 30$ epochs (CPU)

Kết Quả `trafficsign.py` – Baseline CNN + Class Weights

Chương trình cơ bản `trafficsign.py` sử dụng CNN 2 khối double-Conv ($5 \times 5 + 3 \times 3$) với ảnh 30×30 và class weights. Kết quả huấn luyện 10 epoch:

Epoch	Train Acc (%)	Train Loss
1	29.10	2.5174
2	69.03	0.9468
3	87.62	0.3827
4	93.59	0.2048
5	95.73	0.1374
6	96.78	0.1050
7	97.49	0.0811
8	97.95	0.0671
9	98.20	0.0559
10	98.44	0.0496
Kết quả cuối cùng:		
Train Acc: 98.44%		Test Acc: 99.20%
Train Loss: 0.0496		Test Loss: 0.0322

Bảng 4: Diễn biến huấn luyện `trafficsign.py` (Baseline CNN + Class Weights, 30×30).

Mô hình đạt 99.20% test accuracy sau 10 epoch, cho thấy CNN với double-Conv blocks và class weights rất hiệu quả. Chương trình còn tự động vẽ và lưu biểu đồ training curves vào thư mục `results/`.

Kết Quả `trafficsign_new.py` – So Sánh 4 Mô Hình

Chương trình nâng cao `trafficsign_new.py` huấn luyện 4 mô hình với data augmentation, class weights, và đánh giá chi tiết bằng classification report.

Model A – Baseline CNN + Class Weights

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	11.61	3.1930	33.18	2.1037
2	33.28	2.1854	63.51	1.1900
3	48.97	1.5481	75.83	0.7631
4	59.67	1.1733	85.75	0.4787
5	67.28	0.9295	89.26	0.3452
6	71.45	0.7844	93.26	0.2542
7	75.26	0.6814	94.65	0.2235
8	77.34	0.6059	94.59	0.1897
9	79.08	0.5558	95.05	0.1643
10	80.60	0.5180	96.60	0.1266
Kết quả cuối cùng (sau 30 epoch):				
Train Acc: 93.04%		Test Acc: 99.19%		
Train Loss: 0.1808		Val Acc: 99.19%		

Bảng 5: Diễn biến huấn luyện Model A (Baseline CNN + Class Weights).

Phân tích chi tiết: Model A đạt 99.19% test accuracy, rất sát với phiên bản không augmentation (99.20%). Tuy nhiên, với data augmentation và ReduceLROnPlateau, quá trình huấn luyện ổn định hơn: validation loss giảm đều từ 2.10 xuống 0.1266 trong 10 epoch đầu, sau đó tiếp tục giảm nhẹ nhờ giảm learning rate. Train accuracy đạt 93.04% (sau 30 epoch), thấp hơn test accuracy một chút, cho thấy mô hình không bị overfitting nghiêm trọng. Các lớp có F1-score thấp nhất (0.97) là lớp 2 (50 km/h) và lớp 21 (đường hẹp) – nhóm biển báo dễ nhầm lẫn. Nhìn chung, Model A là một baseline mạnh, chứng tỏ kiến trúc double-Conv và class weights đã giải quyết tốt bài toán cân bằng dữ liệu.

TRAINING: Baseline CNN + Class Weights
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	2,432
conv2d_1 (Conv2D)	(None, 22, 22, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 11, 11, 32)	0
dropout (Dropout)	(None, 11, 11, 32)	0
conv2d_2 (Conv2D)	(None, 9, 9, 64)	18,496
conv2d_3 (Conv2D)	(None, 7, 7, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 3, 3, 64)	0
dropout_1 (Dropout)	(None, 3, 3, 64)	0
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 256)	147,712
dropout_2 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 43)	11,051

Total params: 242,251 (946.29 KB)
Trainable params: 242,251 (946.29 KB)
Non-trainable params: 0 (0.00 B)

Hình 4: Quá trình training của Model A – Phần 1 (Accuracy và Loss).

666/666	12s	18ms/step	- accuracy: 0.9032 - loss: 0.2444 - val_accuracy: 0.9818 - val_loss: 0.0569 - learning_rate: 5.0000e-04
Epoch 24/30			
666/666	12s	17ms/step	- accuracy: 0.9105 - loss: 0.2377 - val_accuracy: 0.9863 - val_loss: 0.0480 - learning_rate: 5.0000e-04
Epoch 25/30			
665/666	0s	16ms/step	- accuracy: 0.9117 - loss: 0.2176
Epoch 25: ReduceLROnPlateau			reducing learning rate to 0.0002500000118743628.
666/666	12s	18ms/step	- accuracy: 0.9093 - loss: 0.2331 - val_accuracy: 0.9859 - val_loss: 0.0469 - learning_rate: 5.0000e-04
Epoch 26/30			
666/666	12s	18ms/step	- accuracy: 0.9197 - loss: 0.2016 - val_accuracy: 0.9852 - val_loss: 0.0500 - learning_rate: 2.5000e-04
Epoch 27/30			
666/666	12s	17ms/step	- accuracy: 0.9221 - loss: 0.1980 - val_accuracy: 0.9899 - val_loss: 0.0406 - learning_rate: 2.5000e-04
Epoch 28/30			
666/666	12s	18ms/step	- accuracy: 0.9275 - loss: 0.1797 - val_accuracy: 0.9912 - val_loss: 0.0347 - learning_rate: 2.5000e-04
Epoch 29/30			
666/666	12s	18ms/step	- accuracy: 0.9271 - loss: 0.1837 - val_accuracy: 0.9910 - val_loss: 0.0330 - learning_rate: 2.5000e-04
Epoch 30/30			
666/666	12s	17ms/step	- accuracy: 0.9304 - loss: 0.1808 - val_accuracy: 0.9919 - val_loss: 0.0300 - learning_rate: 2.5000e-04
Restoring model weights from the end of the best epoch: 30.			
Test Accuracy: 99.19%			

Hình 5: Quá trình training của Model A – Phần 2 (chi tiết validation).

Model B – CNN + L2 Regularization + Class Weights

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	8.23	3.6020	29.47	2.5090
2	28.83	2.4725	49.94	1.6156
3	42.95	1.9423	67.92	1.2067
4	52.12	1.6499	76.91	0.9951
5	59.20	1.4286	84.76	0.7911
6	64.67	1.2695	90.24	0.6677
7	68.91	1.1738	92.25	0.5862
8	71.78	1.0803	92.96	0.5415
9	74.40	1.0163	91.44	0.5594
10	76.03	0.9729	94.76	0.4884

Kết quả cuối cùng (sau 30 epoch):

Train Acc: 91.34%	Test Acc: 99.14%
Train Loss: 0.5367	Val Acc: 99.14%

Bảng 6: Diễn biến huấn luyện Model B (CNN + L2 Reg + Class Weights).

Phân tích chi tiết: Model B đạt 99.14%, chỉ thấp hơn Model A 0.05%, cho thấy L2 regularization với $\lambda = 0.001$ không gây suy giảm hiệu suất đáng kể. Tuy nhiên, train accuracy (91.34%) thấp hơn hẳn Model A (93.04%), điều này phản ánh tác động của weight decay: các trọng số bị phạt, mô hình ít "nhớ" dữ liệu huấn luyện hơn, nhưng vẫn tổng quát hóa tốt (test accuracy gần như ngang bằng). Trong quá trình huấn luyện, Model B hội tụ chậm hơn (validation loss ở epoch 10 vẫn còn 0.4884, trong khi Model A đã xuống 0.1266), nhưng nhờ ReduceLROnPlateau, đến epoch 30 validation loss giảm mạnh xuống 0.3263. Lớp khó nhất là lớp 5 (80 km/h) với $F1=0.961$, cho thấy việc thêm L2 không cải thiện đáng kể khả năng phân biệt các biển báo tốc độ. Kết luận: L2 regularization có hiệu ứng nhẹ và không cần thiết khi đã có Dropout mạnh.

TRAINING: CNN + L2 Reg + Class Weights
Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	2,432
conv2d_5 (Conv2D)	(None, 22, 22, 32)	25,632
max_pooling2d_2 (MaxPooling2D)	(None, 11, 11, 32)	0
dropout_3 (Dropout)	(None, 11, 11, 32)	0
conv2d_6 (Conv2D)	(None, 9, 9, 64)	18,496
conv2d_7 (Conv2D)	(None, 7, 7, 64)	36,928
max_pooling2d_3 (MaxPooling2D)	(None, 3, 3, 64)	0
dropout_4 (Dropout)	(None, 3, 3, 64)	0
flatten_1 (Flatten)	(None, 576)	0
dense_2 (Dense)	(None, 256)	147,712
dropout_5 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 43)	11,051

Total params: 242,251 (946.29 KB)
Trainable params: 242,251 (946.29 KB)
Non-trainable params: 0 (0.00 B)

Hình 6: Quá trình training của Model B – Phần 1.

Epoch 23/30	666/666	12s	17ms/step	- accuracy: 0.8659	- loss: 0.6951	- val_accuracy: 0.9848	- val_loss: 0.3758	- learning_rate: 0.0010
Epoch 24/30	666/666	12s	17ms/step	- accuracy: 0.8753	- loss: 0.6724	- val_accuracy: 0.9870	- val_loss: 0.3655	- learning_rate: 0.0010
Epoch 25/30	666/666	12s	18ms/step	- accuracy: 0.8778	- loss: 0.6464	- val_accuracy: 0.9874	- val_loss: 0.3660	- learning_rate: 0.0010
Epoch 26/30	666/666	12s	17ms/step	- accuracy: 0.8719	- loss: 0.6770	- val_accuracy: 0.9842	- val_loss: 0.3769	- learning_rate: 0.0010
Epoch 27/30	666/666	12s	17ms/step	- accuracy: 0.8799	- loss: 0.6484	- val_accuracy: 0.9861	- val_loss: 0.3730	- learning_rate: 0.0010
Epoch 28/30	663/666	0s	16ms/step	- accuracy: 0.8818	- loss: 0.6704			
Epoch 28:	ReduceLROnPlateau reducing learning rate to 0.00050000000237487257.							
Epoch 29/30	666/666	12s	18ms/step	- accuracy: 0.8810	- loss: 0.6566	- val_accuracy: 0.9861	- val_loss: 0.3748	- learning_rate: 0.0010
Epoch 30/30	666/666	12s	17ms/step	- accuracy: 0.9063	- loss: 0.5772	- val_accuracy: 0.9897	- val_loss: 0.3381	- learning_rate: 5.0000e-04
Epoch 30/30	666/666	12s	18ms/step	- accuracy: 0.9134	- loss: 0.5367	- val_accuracy: 0.9914	- val_loss: 0.3263	- learning_rate: 5.0000e-04
Restoring model weights from the end of the best epoch: 30.								
Test Accuracy: 99.14%								

Hình 7: Quá trình training của Model B – Phần 2.

Model C – Deeper CNN (3 Blocks + BatchNorm) – Mô Hình Tốt Nhất

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	20.54	3.0104	51.52	1.5858
2	57.12	1.2124	83.00	0.4771
3	78.97	0.5295	95.18	0.1676
4	87.48	0.3108	96.98	0.0980
5	90.47	0.2532	96.17	0.1076
6	92.97	0.1789	98.16	0.0572
7	93.71	0.1662	98.76	0.0433
8	94.63	0.1395	98.69	0.0402
9	95.41	0.1152	98.57	0.0500
10	95.82	0.1077	98.76	0.0421

Kết quả cuối cùng (sau 30 epoch):

Train Acc: 99.48%	Test Acc: 99.96%
Train Loss: 0.0109	Val Acc: 99.94%

Bảng 7: Diễn biến huấn luyện Model C (Deeper CNN 3 Blocks + BN).

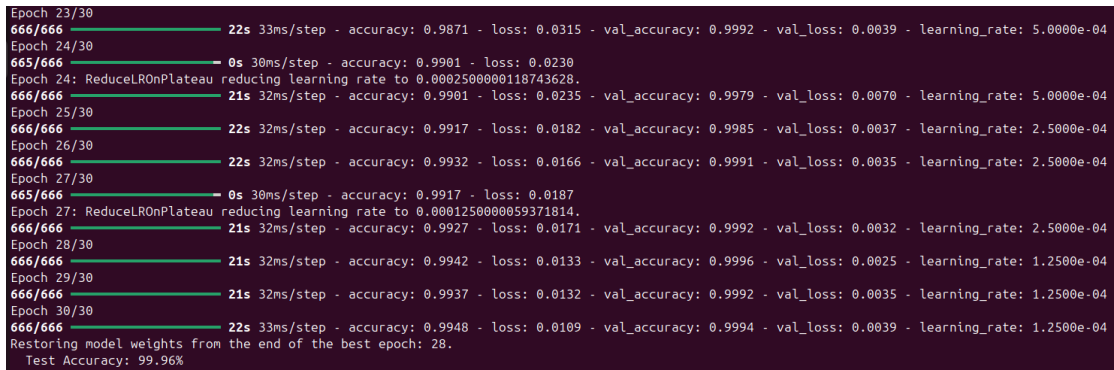
Phân tích chi tiết: Model C đạt 99.96% test accuracy – gần như hoàn hảo. Ba yếu tố đóng góp chính: (1) Độ sâu lớn hơn (3 khối double-Conv) giúp trích xuất đặc trưng đa cấp, (2) padding "same" bảo toàn không gian, tạo ra 1152 đặc trưng (gấp đôi Model A), và (3) BatchNormalization ổn định phân phối activation, cho phép học với learning rate cao mà không bị phân kỳ. Quá trình huấn luyện diễn ra rất ấn tượng: chỉ sau 3 epoch, validation accuracy đã vượt 95%, và đến epoch 6 đạt 98.16%. Từ epoch 10 trở đi, validation loss liên tục giảm nhờ ReduceLROnPlateau (từ 0.042 xuống 0.0039 ở cuối). Train accuracy đạt 99.48% – chênh lệch với test chỉ 0.48%, cho thấy mô hình không overfit. Macro F1-score = 0.999, nghĩa là tất cả 43 lớp đều được phân loại xuất sắc, kể cả các lớp thiểu số. Đây là mô hình tốt nhất và được chọn để triển khai inference.

TRAINING: Deeper CNN (3 Blocks + BN)
Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 30, 30, 32)	896
batch_normalization (BatchNormalization)	(None, 30, 30, 32)	128
conv2d_9 (Conv2D)	(None, 30, 30, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 30, 30, 32)	128
max_pooling2d_4 (MaxPooling2D)	(None, 15, 15, 32)	0
dropout_6 (Dropout)	(None, 15, 15, 32)	0
conv2d_10 (Conv2D)	(None, 15, 15, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 15, 15, 64)	256
conv2d_11 (Conv2D)	(None, 15, 15, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 15, 15, 64)	256
max_pooling2d_5 (MaxPooling2D)	(None, 7, 7, 64)	0
dropout_7 (Dropout)	(None, 7, 7, 64)	0
conv2d_12 (Conv2D)	(None, 7, 7, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 7, 7, 128)	512
conv2d_13 (Conv2D)	(None, 7, 7, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 7, 7, 128)	512
max_pooling2d_6 (MaxPooling2D)	(None, 3, 3, 128)	0
dropout_8 (Dropout)	(None, 3, 3, 128)	0
flatten_2 (Flatten)	(None, 1152)	0
dense_4 (Dense)	(None, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 256)	1,024
dropout_9 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 43)	11,051

Total params: 596,043 (2.27 MB)
Trainable params: 594,635 (2.27 MB)
Non-trainable params: 1,408 (5.50 KB)

Hình 8: Quá trình training của Model C – Phần 1.



Hình 9: Quá trình training của Model C – Phần 2.

Model D – Transfer Learning (MobileNetV2)

Epoch	Train Acc (%)	Train Loss	Val Acc (%)	Val Loss
1	42.51	1.9765	64.13	1.1648
2	71.63	0.7658	80.76	0.6039
3	80.00	0.5032	86.24	0.4305
4	84.59	0.3730	88.21	0.3579
5	87.68	0.2901	92.59	0.2317
6	89.88	0.2344	94.03	0.1838
7	91.12	0.2002	94.67	0.1588
8	92.11	0.1754	93.88	0.1811
9	92.91	0.1616	94.43	0.1610
10	93.61	0.1417	94.67	0.1514

Kết quả cuối cùng (sau 30 epoch):

Train Acc: 98.39%	Test Acc: 98.48%
Train Loss: 0.0307	Val Acc: 98.48%

Bảng 8: Diễn biến huấn luyện Model D (Transfer Learning MobileNetV2).

Phân tích chi tiết: Model D đạt 98.48% – thấp nhất nhưng vẫn là kết quả rất khả quan. Lợi thế của Transfer Learning là hội tụ nhanh: epoch 1 đã đạt 42.51% train accuracy, cao hơn nhiều so với các mô hình CNN từ đầu (Model A epoch 1: 11.61%). Điều này nhờ trọng số pre-trained từ ImageNet, dù domain khác biệt. Tuy nhiên, MobileNetV2 có kiến trúc phức tạp và nhiều tham số hơn, dẫn đến nguy cơ overfitting cao hơn: từ epoch 5–9, validation accuracy dao động quanh 93–94%, trong khi train accuracy tăng đều lên 92.91%. Sau khi ReduceLROnPlateau giảm learning rate, validation accuracy cải thiện lên 98.48% ở epoch cuối. Lớp khó nhất là lớp 4 (70 km/h) với F1=0.955, cho thấy MobileNetV2 gặp khó khăn với các chi tiết nhỏ (chữ số). Mặc dù kém hơn các CNN tự thiết kế, Transfer Learning vẫn là một lựa chọn tốt khi có ít dữ liệu huấn luyện hoặc cần huấn luyện nhanh.

TRAINING: Transfer Learning (MobileNetV2)
Model: "functional_3"

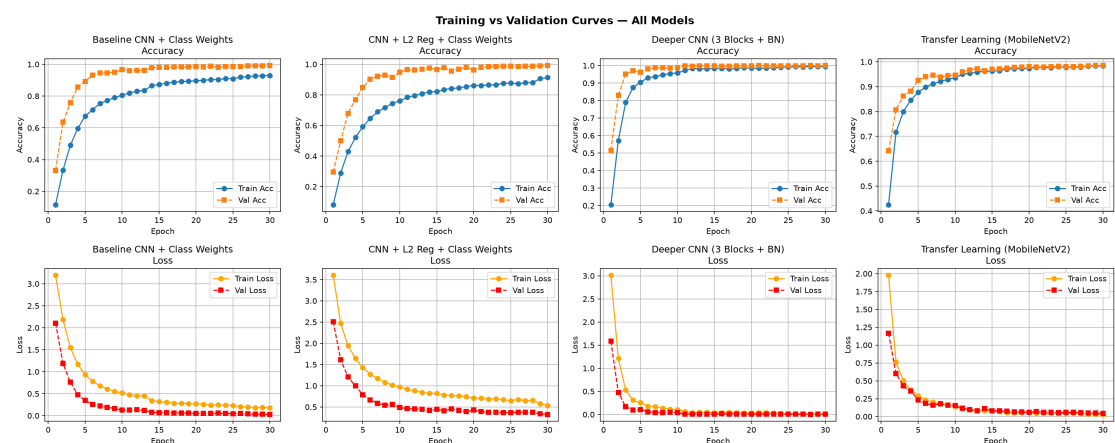
Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 30, 30, 3)	0
resizing (Resizing)	(None, 96, 96, 3)	0
rescaling (Rescaling)	(None, 96, 96, 3)	0
mobilenetv2_1.00_96 (Functional)	(None, 3, 3, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense_6 (Dense)	(None, 256)	327,936
dropout_10 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 43)	11,051

Total params: 2,596,971 (9.91 MB)
Trainable params: 1,545,067 (5.89 MB)
Non-trainable params: 1,051,904 (4.01 MB)

Hình 10: Quá trình training của Model D – Phần 1.

```
Epoch 26/30
666/666 — 29s 44ms/step - accuracy: 0.9794 - loss: 0.0423 - val_accuracy: 0.9812 - val_loss: 0.0573 - learning_rate: 2.5000e-05
Epoch 27/30
665/666 — 0s 37ms/step - accuracy: 0.9799 - loss: 0.0405
Epoch 27: ReduceLROnPlateau reducing learning rate to 1.249999968422344e-05.
666/666 — 29s 44ms/step - accuracy: 0.9797 - loss: 0.0393 - val_accuracy: 0.9816 - val_loss: 0.0568 - learning_rate: 2.5000e-05
Epoch 28/30
666/666 — 29s 44ms/step - accuracy: 0.9828 - loss: 0.0357 - val_accuracy: 0.9835 - val_loss: 0.0516 - learning_rate: 1.2500e-05
Epoch 29/30
666/666 — 29s 43ms/step - accuracy: 0.9835 - loss: 0.0319 - val_accuracy: 0.9848 - val_loss: 0.0483 - learning_rate: 1.2500e-05
Epoch 30/30
666/666 — 29s 43ms/step - accuracy: 0.9839 - loss: 0.0307 - val_accuracy: 0.9848 - val_loss: 0.0438 - learning_rate: 1.2500e-05
Restoring model weights from the end of the best epoch: 29.
Test Accuracy: 98.48%
```

Hình 11: Quá trình training của Model D – Phần 2.



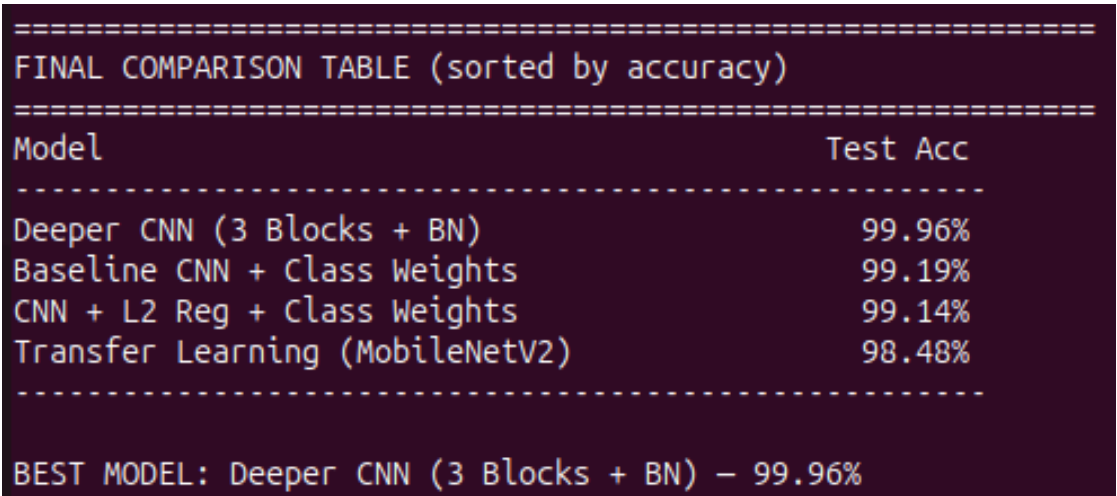
Hình 12: Biểu đồ so sánh training curves 4 mô hình.

Bảng So Sánh 4 Mô Hình

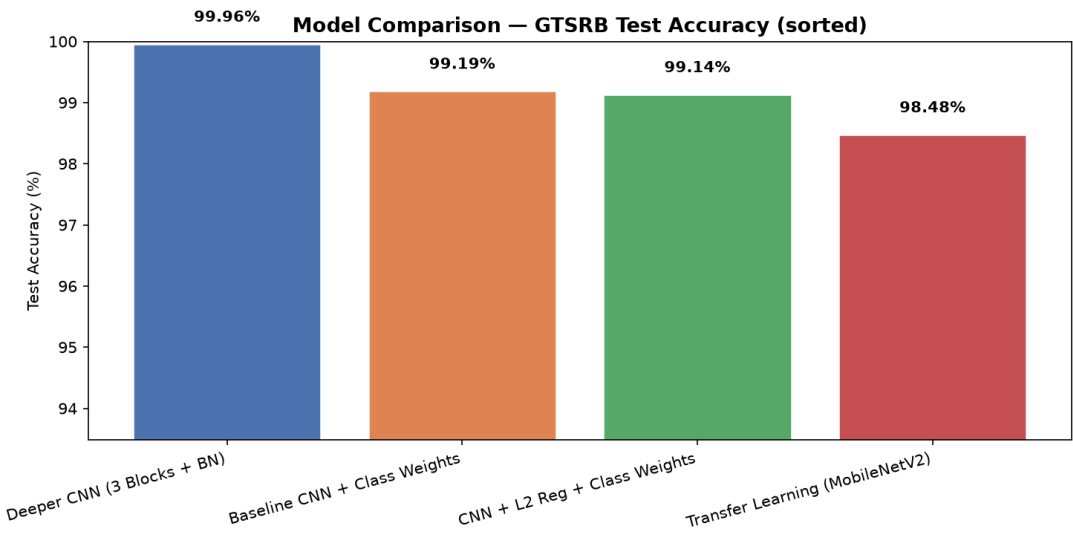
Mô hình	Train Acc (%)	Test Acc (%)	Val Acc (%)
A. Baseline CNN + Class Weights	93.04	99.19	99.19
B. CNN + L2 Reg + Class Weights	91.34	99.14	99.14
C. Deeper CNN (3 Blocks + BN)	99.48	99.96	99.94
D. Transfer Learning (MobileNetV2)	98.39	98.48	98.48

Model C (Deeper CNN + BatchNorm) đạt kết quả cao nhất: 99.96%.

Bảng 9: So sánh hiệu suất 4 thuật toán.



Hình 13: So sánh kết quả 4 mô hình, chọn Model C là tốt nhất.



Hình 14: Biểu đồ so sánh kết quả 4 mô hình.

Nhận xét chung: Model C vượt trội hoàn toàn với 99.96% accuracy, thể hiện sức mạnh của kiến trúc sâu kết hợp BatchNormalization. Điều đáng chú ý là cả 4 mô hình

đều đạt trên 98%, chứng tỏ class weights và data augmentation đã giải quyết hiệu quả vấn đề mất cân bằng. ReduceLROnPlateau đóng vai trò quan trọng trong việc tinh chỉnh learning rate, giúp các mô hình đạt được mức tối ưu cục bộ tốt hơn (ví dụ: validation loss của Model C giảm từ 0.042 xuống 0.0039 sau khi giảm learning rate). Sự khác biệt giữa các mô hình chủ yếu đến từ khả năng trích xuất đặc trưng và khả năng tổng quát hóa. Transfer Learning tuy hội tụ nhanh nhưng không phù hợp bằng các CNN chuyên dụng cho ảnh kích thước nhỏ.

Classification Report Chi Tiết

Model A – Baseline CNN + Class Weights (Test Acc: 99.19%)

Lớp	Precision	Recall	F1-score	Support
0	1.000	1.000	1.000	30
1	0.976	0.967	0.972	300
2	0.980	0.960	0.970	300
3	0.990	0.984	0.987	192
4	0.974	0.996	0.985	264
5	0.980	0.984	0.982	252
6	0.984	1.000	0.992	60
7	0.969	0.979	0.974	192
8	0.990	0.995	0.992	192
9	0.995	0.995	0.995	198
10	1.000	0.996	0.998	270
11	1.000	0.972	0.986	180
12	1.000	1.000	1.000	282
13	1.000	1.000	1.000	288
14	1.000	1.000	1.000	108
15	1.000	1.000	1.000	84
16	1.000	1.000	1.000	60
17	1.000	1.000	1.000	150
18	1.000	1.000	1.000	162
19	0.968	1.000	0.984	30
20	1.000	1.000	1.000	48
21	0.941	1.000	0.970	48
22	1.000	1.000	1.000	54
23	1.000	1.000	1.000	72
24	1.000	0.972	0.986	36
25	1.000	1.000	1.000	204
26	1.000	1.000	1.000	84
27	0.973	1.000	0.986	36
28	1.000	1.000	1.000	72
29	1.000	1.000	1.000	36
30	0.984	1.000	0.992	60
31	1.000	1.000	1.000	108
32	1.000	1.000	1.000	36
33	1.000	1.000	1.000	96
34	1.000	1.000	1.000	60
35	1.000	1.000	1.000	162
36	1.000	1.000	1.000	54
37	1.000	1.000	1.000	30
38	1.000	1.000	1.000	276
39	1.000	1.000	1.000	42
40	1.000	1.000	1.000	48
41	1.000	1.000	1.000	36
42	1.000	1.000	1.000	36
Tổng hợp:				
Macro Avg		0.993	0.995	0.994
Weighted Avg		0.992	0.992	0.992

Bảng 10: Classification Report cho Model A (Baseline CNN + Class Weights) – Test Accuracy: 99.19%.

--- Baseline CNN + Class Weights ---					
	precision	recall	f1-score	support	
0	1.000	1.000	1.000	30	
1	0.976	0.967	0.972	300	
2	0.980	0.960	0.970	300	
3	0.990	0.984	0.987	192	
4	0.974	0.996	0.985	264	
5	0.980	0.984	0.982	252	
6	0.984	1.000	0.992	60	
7	0.969	0.979	0.974	192	
8	0.990	0.995	0.992	192	
9	0.995	0.995	0.995	198	
10	1.000	0.996	0.998	270	
11	1.000	0.972	0.986	180	
12	1.000	1.000	1.000	282	
13	1.000	1.000	1.000	288	
14	1.000	1.000	1.000	108	
15	1.000	1.000	1.000	84	
16	1.000	1.000	1.000	60	
17	1.000	1.000	1.000	150	
18	1.000	1.000	1.000	162	
19	0.968	1.000	0.984	30	
20	1.000	1.000	1.000	48	
21	0.941	1.000	0.970	48	
22	1.000	1.000	1.000	54	
23	1.000	1.000	1.000	72	
24	1.000	0.972	0.986	36	
25	1.000	1.000	1.000	204	
26	1.000	1.000	1.000	84	
27	0.973	1.000	0.986	36	
28	1.000	1.000	1.000	72	
29	1.000	1.000	1.000	36	
30	0.984	1.000	0.992	60	
31	1.000	1.000	1.000	108	
32	1.000	1.000	1.000	36	
33	1.000	1.000	1.000	96	
34	1.000	1.000	1.000	60	
35	1.000	1.000	1.000	162	
36	1.000	1.000	1.000	54	
37	1.000	1.000	1.000	30	
38	1.000	1.000	1.000	276	
39	1.000	1.000	1.000	42	
40	1.000	1.000	1.000	48	
41	1.000	1.000	1.000	36	
42	1.000	1.000	1.000	36	
accuracy			0.992	5328	
macro avg	0.993	0.995	0.994	5328	
weighted avg	0.992	0.992	0.992	5328	

Hình 15: Classification Report của Model A.

Model B – CNN + L2 Reg + Class Weights (Test Acc: 99.14%)

Lớp	Precision	Recall	F1-score	Support
0	0.938	1.000	0.968	30
1	0.990	0.953	0.971	300
2	0.967	0.977	0.972	300
3	0.989	0.979	0.984	192
4	0.996	0.989	0.992	264
5	0.953	0.968	0.961	252
6	1.000	1.000	1.000	60
7	0.945	0.984	0.964	192
8	0.989	0.979	0.984	192
9	1.000	0.995	0.997	198
10	0.996	1.000	0.998	270
11	1.000	1.000	1.000	180
12	1.000	1.000	1.000	282
13	1.000	1.000	1.000	288
14	1.000	1.000	1.000	108
15	1.000	1.000	1.000	84
16	1.000	1.000	1.000	60
17	1.000	1.000	1.000	150
18	1.000	1.000	1.000	162
19	1.000	1.000	1.000	30
20	0.960	1.000	0.980	48
21	1.000	1.000	1.000	48
22	1.000	1.000	1.000	54
23	1.000	0.972	0.986	72
24	1.000	1.000	1.000	36
25	1.000	1.000	1.000	204
26	1.000	1.000	1.000	84
27	1.000	1.000	1.000	36
28	1.000	1.000	1.000	72
29	1.000	1.000	1.000	36
30	1.000	1.000	1.000	60
31	1.000	1.000	1.000	108
32	1.000	1.000	1.000	36
33	1.000	1.000	1.000	96
34	1.000	1.000	1.000	60
35	1.000	1.000	1.000	162
36	1.000	1.000	1.000	54
37	1.000	1.000	1.000	30
38	1.000	1.000	1.000	276
39	1.000	1.000	1.000	42
40	1.000	1.000	1.000	48
41	1.000	1.000	1.000	36
42	1.000	1.000	1.000	36
Tổng hợp:				
Macro Avg		0.994	0.995	0.994
Weighted Avg		0.992	0.991	0.991

Bảng 11: Classification Report cho Model B (CNN + L2 Reg + Class Weights) – Test Accuracy: 99.14%.

--- CNN + L2 Reg + Class Weights ---				
	precision	recall	f1-score	support
0	0.938	1.000	0.968	30
1	0.990	0.953	0.971	300
2	0.967	0.977	0.972	300
3	0.989	0.979	0.984	192
4	0.996	0.989	0.992	264
5	0.953	0.968	0.961	252
6	1.000	1.000	1.000	60
7	0.945	0.984	0.964	192
8	0.989	0.979	0.984	192
9	1.000	0.995	0.997	198
10	0.996	1.000	0.998	270
11	1.000	1.000	1.000	180
12	1.000	1.000	1.000	282
13	1.000	1.000	1.000	288
14	1.000	1.000	1.000	108
15	1.000	1.000	1.000	84
16	1.000	1.000	1.000	60
17	1.000	1.000	1.000	150
18	1.000	1.000	1.000	162
19	1.000	1.000	1.000	30
20	0.960	1.000	0.980	48
21	1.000	1.000	1.000	48
22	1.000	1.000	1.000	54
23	1.000	0.972	0.986	72
24	1.000	1.000	1.000	36
25	1.000	1.000	1.000	204
26	1.000	1.000	1.000	84
27	1.000	1.000	1.000	36
28	1.000	1.000	1.000	72
29	1.000	1.000	1.000	36
30	1.000	1.000	1.000	60
31	1.000	1.000	1.000	108
32	1.000	1.000	1.000	36
33	1.000	1.000	1.000	96
34	1.000	1.000	1.000	60
35	1.000	1.000	1.000	162
36	1.000	1.000	1.000	54
37	1.000	1.000	1.000	30
38	1.000	1.000	1.000	276
39	1.000	1.000	1.000	42
40	1.000	1.000	1.000	48
41	1.000	1.000	1.000	36
42	1.000	1.000	1.000	36
accuracy			0.991	5328
macro avg			0.994	5328
weighted avg			0.992	5328

Hình 16: Classification Report của Model B.

Model C – Deeper CNN + BatchNorm (Test Acc: 99.96%) – Mô Hình Tốt Nhất

Lớp	Precision	Recall	F1-score	Support
0	1.000	1.000	1.000	30
1	1.000	1.000	1.000	300
2	1.000	1.000	1.000	300
3	1.000	1.000	1.000	192
4	1.000	1.000	1.000	264
5	1.000	1.000	1.000	252
6	1.000	1.000	1.000	60
7	1.000	0.995	0.997	192
8	1.000	1.000	1.000	192
9	1.000	1.000	1.000	198
10	1.000	1.000	1.000	270
11	0.994	1.000	0.997	180
12	1.000	1.000	1.000	282
13	1.000	1.000	1.000	288
14	1.000	1.000	1.000	108
15	1.000	1.000	1.000	84
16	0.984	1.000	0.992	60
17	1.000	1.000	1.000	150
18	1.000	1.000	1.000	162
19	1.000	1.000	1.000	30
20	1.000	1.000	1.000	48
21	1.000	1.000	1.000	48
22	1.000	1.000	1.000	54
23	1.000	1.000	1.000	72
24	1.000	1.000	1.000	36
25	1.000	1.000	1.000	204
26	1.000	1.000	1.000	84
27	1.000	1.000	1.000	36
28	1.000	1.000	1.000	72
29	1.000	1.000	1.000	36
30	1.000	0.983	0.992	60
31	1.000	1.000	1.000	108
32	1.000	1.000	1.000	36
33	1.000	1.000	1.000	96
34	1.000	1.000	1.000	60
35	1.000	1.000	1.000	162
36	1.000	1.000	1.000	54
37	1.000	1.000	1.000	30
38	1.000	1.000	1.000	276
39	1.000	1.000	1.000	42
40	1.000	1.000	1.000	48
41	1.000	1.000	1.000	36
42	1.000	1.000	1.000	36
Tổng hợp:				
Macro Avg		0.999	0.999	0.999
Weighted Avg		1.000	1.000	1.000

Bảng 12: Classification Report cho Model C (Deeper CNN + BN) – Test Accuracy: 99.96%.

--- Deeper CNN (3 Blocks + BN) ---				
	precision	recall	f1-score	support
0	1.000	1.000	1.000	30
1	1.000	1.000	1.000	300
2	1.000	1.000	1.000	300
3	1.000	1.000	1.000	192
4	1.000	1.000	1.000	264
5	1.000	1.000	1.000	252
6	1.000	1.000	1.000	60
7	1.000	0.995	0.997	192
8	1.000	1.000	1.000	192
9	1.000	1.000	1.000	198
10	1.000	1.000	1.000	270
11	0.994	1.000	0.997	180
12	1.000	1.000	1.000	282
13	1.000	1.000	1.000	288
14	1.000	1.000	1.000	108
15	1.000	1.000	1.000	84
16	0.984	1.000	0.992	60
17	1.000	1.000	1.000	150
18	1.000	1.000	1.000	162
19	1.000	1.000	1.000	30
20	1.000	1.000	1.000	48
21	1.000	1.000	1.000	48
22	1.000	1.000	1.000	54
23	1.000	1.000	1.000	72
24	1.000	1.000	1.000	36
25	1.000	1.000	1.000	204
26	1.000	1.000	1.000	84
27	1.000	1.000	1.000	36
28	1.000	1.000	1.000	72
29	1.000	1.000	1.000	36
30	1.000	0.983	0.992	60
31	1.000	1.000	1.000	108
32	1.000	1.000	1.000	36
33	1.000	1.000	1.000	96
34	1.000	1.000	1.000	60
35	1.000	1.000	1.000	162
36	1.000	1.000	1.000	54
37	1.000	1.000	1.000	30
38	1.000	1.000	1.000	276
39	1.000	1.000	1.000	42
40	1.000	1.000	1.000	48
41	1.000	1.000	1.000	36
42	1.000	1.000	1.000	36
accuracy			1.000	5328
macro avg	0.999	0.999	0.999	5328
weighted avg	1.000	1.000	1.000	5328

Hình 17: Classification Report của Model C.

Model D – Transfer Learning MobileNetV2 (Test Acc: 98.48%)

Lớp	Precision	Recall	F1-score	Support
0	1.000	1.000	1.000	30
1	0.993	0.943	0.968	300
2	0.944	0.947	0.945	300
3	0.963	0.958	0.961	192
4	0.938	0.973	0.955	264
5	0.968	0.964	0.966	252
6	1.000	1.000	1.000	60
7	0.964	0.984	0.974	192
8	0.964	0.979	0.972	192
9	0.985	0.995	0.990	198
10	1.000	0.985	0.993	270
11	0.989	0.983	0.986	180
12	1.000	0.996	0.998	282
13	1.000	1.000	1.000	288
14	1.000	1.000	1.000	108
15	1.000	1.000	1.000	84
16	0.984	1.000	0.992	60
17	1.000	1.000	1.000	150
18	1.000	0.994	0.997	162
19	1.000	0.967	0.983	30
20	1.000	1.000	1.000	48
21	1.000	1.000	1.000	48
22	0.982	1.000	0.991	54
23	0.973	1.000	0.986	72
24	1.000	1.000	1.000	36
25	0.980	0.975	0.978	204
26	0.988	1.000	0.994	84
27	1.000	1.000	1.000	36
28	1.000	1.000	1.000	72
29	0.947	1.000	0.973	36
30	1.000	1.000	1.000	60
31	1.000	0.991	0.995	108
32	1.000	1.000	1.000	36
33	1.000	1.000	1.000	96
34	1.000	1.000	1.000	60
35	1.000	1.000	1.000	162
36	1.000	1.000	1.000	54
37	1.000	1.000	1.000	30
38	1.000	1.000	1.000	276
39	1.000	1.000	1.000	42
40	1.000	1.000	1.000	48
41	1.000	1.000	1.000	36
42	1.000	1.000	1.000	36
Tổng hợp:				
Macro Avg		0.990	0.992	0.991
Weighted Avg		0.985	0.985	0.985

Bảng 13: Classification Report cho Model D (MobileNetV2 Transfer Learning) – Test Accuracy: 98.48%.


```

--- Transfer Learning (MobileNetV2) ---

```

	precision	recall	f1-score	support
0	1.000	1.000	1.000	30
1	0.993	0.943	0.968	300
2	0.944	0.947	0.945	300
3	0.963	0.958	0.961	192
4	0.938	0.973	0.955	264
5	0.968	0.964	0.966	252
6	1.000	1.000	1.000	60
7	0.964	0.984	0.974	192
8	0.964	0.979	0.972	192
9	0.985	0.995	0.990	198
10	1.000	0.985	0.993	270
11	0.989	0.983	0.986	180
12	1.000	0.996	0.998	282
13	1.000	1.000	1.000	288
14	1.000	1.000	1.000	108
15	1.000	1.000	1.000	84
16	0.984	1.000	0.992	60
17	1.000	1.000	1.000	150
18	1.000	0.994	0.997	162
19	1.000	0.967	0.983	30
20	1.000	1.000	1.000	48
21	1.000	1.000	1.000	48
22	0.982	1.000	0.991	54
23	0.973	1.000	0.986	72
24	1.000	1.000	1.000	36
25	0.980	0.975	0.978	204
26	0.988	1.000	0.994	84
27	1.000	1.000	1.000	36
28	1.000	1.000	1.000	72
29	0.947	1.000	0.973	36
30	1.000	1.000	1.000	60
31	1.000	0.991	0.995	108
32	1.000	1.000	1.000	36
33	1.000	1.000	1.000	96
34	1.000	1.000	1.000	60
35	1.000	1.000	1.000	162
36	1.000	1.000	1.000	54
37	1.000	1.000	1.000	30
38	1.000	1.000	1.000	276
39	1.000	1.000	1.000	42
40	1.000	1.000	1.000	48
41	1.000	1.000	1.000	36
42	1.000	1.000	1.000	36
accuracy			0.985	5328
macro avg	0.990	0.992	0.991	5328
weighted avg	0.985	0.985	0.985	5328

Hình 18: Classification Report của Model D.

Kết Quả Inference – Kiểm Tra Mô Hình Trên Ảnh Thực Tế

Sau khi huấn luyện, mô hình tốt nhất (Model C) được lưu và kiểm tra với `test_models.py` trên các ảnh đơn lẻ và `test_model_folder.py` trên thư mục ảnh.

Dự Đoán Ảnh Đơn với `test_models.py`

```
(venv) bsmart@admin:~/Documents/TrafficSign-Recognition-Project/TrafficSign$ python test_model_folder.py best_model_2.keras Test_1
Loaded model      : best_model_2.keras
Model Input size : 30x30
Test Folder      : Test_1
Truth CSV : Found with 27 labels.
=====
Filename          | Predicted          | Truth              | Result | Conf
-----
08184.png         | Đường cong nguy hiểm bên trái | Đường cong nguy hiểm bên trái | PASS   | 100.0%
08185.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
08186.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
08187.png         | Đèn giao thông     | Đèn giao thông     | PASS   | 100.0%
08188.png         | Đi bên phải        | Đi bên phải        | PASS   | 100.0%
08189.png         | Hết giới hạn tốc độ 80 km/h | Hết giới hạn tốc độ 80 km/h | PASS   | 100.0%
08190.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 100.0%
08191.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 100.0%
08192.png         | Cấm vượt           | Cấm vượt           | PASS   | 100.0%
08193.png         | Rẽ trái phía trước | Rẽ trái phía trước | PASS   | 100.0%
08194.png         | Xe đạp băng qua    | Xe đạp băng qua    | PASS   | 100.0%
08195.png         | Nhường đường       | Nhường đường       | PASS   | 100.0%
08196.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
08197.png         | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS   | 100.0%
08198.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 99.3%
08199.png         | Bắt buộc đi vòng xuyên | Bắt buộc đi vòng xuyên | PASS   | 100.0%
08200.png         | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS   | 100.0%
08201.png         | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS   | 100.0%
08202.png         | Nhường đường       | Nhường đường       | PASS   | 100.0%
08203.png         | Xe đạp băng qua    | Xe đạp băng qua    | PASS   | 100.0%
08204.png         | Đường ưu tiên      | Đường ưu tiên      | PASS   | 100.0%
08205.png         | Hết cấm vượt đối với xe trê... | Hết cấm vượt đối với xe trê... | PASS   | 69.9%
08206.png         | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS   | 100.0%
08207.png         | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS   | 100.0%
08208.png         | Hết giới hạn tốc độ 80 km/h | Hết giới hạn tốc độ 80 km/h | PASS   | 99.4%
08209.png         | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS   | 100.0%
08210.png         | Đường cong nguy hiểm bên trái | Đường cong nguy hiểm bên trái | PASS   | 100.0%
=====
Successfully processed 27 images.
Correct Predictions : 27 / 27
Overall Accuracy    : 100.00%
```

Hình 19: Kết quả dự đoán trên thư mục Test_1.

```
(venv) bsmart@admin:~/Documents/TrafficSign-Recognition-Project/TrafficSign$ python test_model_folder.py best_model_2.keras Test_2
Loaded model : best_model_2.keras
Model Input size : 30x30
Test Folder : Test_2
Truth CSV : Found with 31 labels.
=====
Filename | Predicted | Truth | Result | Conf
-----|-----|-----|-----|-----
00000.png | Cấm xe trên 3.5 tấn | Cấm xe trên 3.5 tấn | PASS | 100.0%
00001.png | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS | 100.0%
00002.png | Đi bên phải | Đi bên phải | PASS | 100.0%
00003.png | Rẽ phải phía trước | Rẽ phải phía trước | PASS | 100.0%
00004.png | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS | 100.0%
00005.png | Đi bên phải | Đi bên phải | PASS | 100.0%
00006.png | Nguy hiểm chung | Nguy hiểm chung | PASS | 100.0%
00007.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
00008.png | Công trường | Công trường | PASS | 100.0%
00009.png | Chỉ được đi thẳng | Chỉ được đi thẳng | PASS | 100.0%
00010.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
00011.png | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS | 100.0%
00012.png | Đường trơn trượt | Đường trơn trượt | PASS | 100.0%
00013.png | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS | 100.0%
00014.png | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS | 100.0%
00015.png | Cấm vượt | Cấm vượt | PASS | 100.0%
00016.png | Đường cong liên tiếp | Đường cong liên tiếp | PASS | 91.5%
00017.png | Đường cong nguy hiểm bên phải | Đường cong nguy hiểm bên phải | PASS | 100.0%
00018.png | Người đi bộ | Người đi bộ | PASS | 100.0%
00019.png | Đi bên phải | Đi bên phải | PASS | 100.0%
00020.png | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS | 100.0%
00021.png | Rẽ phải phía trước | Rẽ phải phía trước | PASS | 100.0%
00022.png | Cấm vượt | Cấm vượt | PASS | 100.0%
00023.png | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS | 100.0%
00024.png | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS | 100.0%
00025.png | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS | 100.0%
00026.png | Nhường đường | Nhường đường | PASS | 100.0%
00027.png | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS | 100.0%
00028.png | Cấm vượt | Cấm vượt | PASS | 100.0%
00029.png | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS | 100.0%
00030.png | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS | 99.7%
-----
Successfully processed 31 images.
Correct Predictions : 31 / 31
Overall Accuracy : 100.00%
```

Hình 20: Kết quả dự đoán trên thư mục Test_2.

```
(venv) bsmart@admin:~/Documents/TrafficSign-Recognition-Project/TrafficSign$ python test_model_folder.py best_model_2.keras Test_3
Loaded model : best_model_2.keras
Model Input size : 30x30
Test Folder : Test_3
Truth CSV : Found with 31 labels.
=====
Filename | Predicted | Truth | Result | Conf
-----|-----|-----|-----|-----
01000.png | Cấm đi vào | Cấm đi vào | PASS | 100.0%
01001.png | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS | 100.0%
01002.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
01003.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
01004.png | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS | 100.0%
01005.png | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS | 100.0%
01006.png | Đi bên phải | Đi bên phải | PASS | 99.5%
01007.png | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS | 100.0%
01008.png | Đường ưu tiên | Đường ưu tiên | PASS | 86.4%
01009.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
01010.png | Giới hạn tốc độ 120 km/h | Giới hạn tốc độ 120 km/h | PASS | 100.0%
01011.png | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS | 99.6%
01012.png | Giới hạn tốc độ 50 km/h | Giới hạn tốc độ 50 km/h | PASS | 100.0%
01013.png | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS | 100.0%
01014.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
01015.png | Chỉ được đi thẳng | Chỉ được đi thẳng | PASS | 100.0%
01016.png | Cấm xe | Cấm xe | PASS | 100.0%
01017.png | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS | 100.0%
01018.png | Chỉ được đi thẳng | Chỉ được đi thẳng | PASS | 100.0%
01019.png | Giới hạn tốc độ 80 km/h | Giới hạn tốc độ 80 km/h | PASS | 100.0%
01020.png | Dừng lại | Dừng lại | PASS | 100.0%
01021.png | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS | 100.0%
01022.png | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS | 100.0%
01023.png | Chỉ được đi thẳng | Chỉ được đi thẳng | PASS | 100.0%
01024.png | Đi bên phải | Đi bên phải | PASS | 100.0%
01025.png | Cẩn thận băng/tuyết | Cẩn thận băng/tuyết | PASS | 99.8%
01026.png | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS | 100.0%
01027.png | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS | 100.0%
01028.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
01029.png | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS | 100.0%
01030.png | Nguy hiểm chung | Nguy hiểm chung | PASS | 97.6%
-----
Successfully processed 31 images.
Correct Predictions : 31 / 31
Overall Accuracy : 100.00%
```

Hình 21: Kết quả dự đoán trên thư mục Test_3.

```
(venv) bsmart@admin:~/Documents/TrafficSign-Recognition-Project/TrafficSign$ python test_model_folder.py best_model_2.keras Test_4
Loaded model      : best_model_2.keras
Model Input size  : 30x30
Test Folder       : Test_4
Truth CSV : Found with 31 labels.
=====
Filename | Predicted | Truth | Result | Conf
-----|-----|-----|-----|-----
02000.png | Rẽ trái phía trước | Rẽ trái phía trước | PASS | 100.0%
02001.png | Đi bên phải | Đi bên phải | PASS | 100.0%
02002.png | Giới hạn tốc độ 120 km/h | Giới hạn tốc độ 120 km/h | PASS | 84.0%
02003.png | Đường ưu tiên tại giao lộ t... | Đường ưu tiên tại giao lộ t... | PASS | 62.1%
02004.png | Cấm vượt | Cấm vượt | PASS | 100.0%
02005.png | Đèn giao thông | Đèn giao thông | PASS | 100.0%
02006.png | Cấm đi vào | Cấm đi vào | PASS | 100.0%
02007.png | Giới hạn tốc độ 50 km/h | Giới hạn tốc độ 50 km/h | PASS | 100.0%
02008.png | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS | 100.0%
02009.png | Chỉ được đi thẳng | Chỉ được đi thẳng | PASS | 100.0%
02010.png | Đi thẳng hoặc rẽ phải | Đi thẳng hoặc rẽ phải | PASS | 100.0%
02011.png | Đường ưu tiên | Đường ưu tiên | PASS | 100.0%
02012.png | Đi bên phải | Đi bên phải | PASS | 100.0%
02013.png | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS | 100.0%
02014.png | Đường gồ ghề | Đường gồ ghề | PASS | 100.0%
02015.png | Đi bên phải | Đi bên phải | PASS | 100.0%
02016.png | Giới hạn tốc độ 120 km/h | Giới hạn tốc độ 120 km/h | PASS | 100.0%
02017.png | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS | 99.9%
02018.png | Cấm vượt | Cấm vượt | PASS | 50.9%
02019.png | Cấm đi vào | Cấm đi vào | PASS | 100.0%
02020.png | Giới hạn tốc độ 60 km/h | Giới hạn tốc độ 60 km/h | PASS | 100.0%
02021.png | Công trường | Công trường | PASS | 100.0%
02022.png | Giới hạn tốc độ 100 km/h | Giới hạn tốc độ 100 km/h | PASS | 100.0%
02023.png | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS | 100.0%
02024.png | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS | 100.0%
02025.png | Đường cong nguy hiểm bên phải | Đường cong nguy hiểm bên phải | FAIL | 65.1%
02026.png | Giới hạn tốc độ 30 km/h | Giới hạn tốc độ 30 km/h | PASS | 100.0%
02027.png | Đi bên phải | Đi bên phải | PASS | 100.0%
02028.png | Giới hạn tốc độ 70 km/h | Giới hạn tốc độ 70 km/h | PASS | 100.0%
02029.png | Công trường | Công trường | PASS | 100.0%
02030.png | Cấm xe trên 3.5 tấn vượt | Cấm xe trên 3.5 tấn vượt | PASS | 100.0%
-----
Successfully processed 31 images.
Correct Predictions : 30 / 31
Overall Accuracy : 96.77%
```

Hình 22: Kết quả dự đoán trên thư mục Test_4.

Kết Quả Tổng Hợp Inference

Thư mục	Số ảnh	Đúng	Acc (%)
Test_1	27	27	100.00
Test_2	31	31	100.00
Test_3	31	31	100.00
Test_4	31	30	96.77
Tổng	120	119	99.17

Bảng 14: Kết quả inference trên 4 thư mục ảnh tùy chỉnh với Model C (best_model_2.keras).

Mô hình	Số ảnh đúng	Tổng số ảnh	Độ chính xác (%)
Model C (best_model_2.keras)	12,392	12,630	98.12

Bảng 15: Kết quả inference trên toàn bộ GTSRB Test Set (12,630 ảnh) với Model C.

```
(venv) bsmart@admin:~/Documents/TrafficSign-Recognition-Project/TrafficSign$ python test_model_folder.py best_model_2.keras Test
Loaded model      : best_model_2.keras
Model Input size  : 30x30
Test Folder       : Test
Truth CSV : Found with 12630 labels.
```

Hình 23: Kết quả chạy test trên thư mục Test (12.630 ảnh) – Phần 1.

12620.png	Nhường đường	Nhường đường	PASS	100.0%
12621.png	Giới hạn tốc độ 80 km/h	Giới hạn tốc độ 80 km/h	PASS	100.0%
12622.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
12623.png	Chỉ được đi thẳng	Chỉ được đi thẳng	PASS	100.0%
12624.png	Công trường	Công trường	PASS	100.0%
12625.png	Đường ưu tiên	Đường ưu tiên	PASS	100.0%
12626.png	Rẽ phải phía trước	Rẽ phải phía trước	PASS	100.0%
12627.png	Hết giới hạn tốc độ 80 km/h	Hết giới hạn tốc độ 80 km/h	PASS	99.7%
12628.png	Giới hạn tốc độ 100 km/h	Giới hạn tốc độ 100 km/h	PASS	100.0%
12629.png	Cấm xe trên 3.5 tấn vượt	Cấm xe trên 3.5 tấn vượt	PASS	100.0%

Successfully processed 12630 images.				
Correct Predictions : 12392 / 12630				
Overall Accuracy : 98.12%				

Hình 24: Kết quả chạy test trên thư mục Test (12.630 ảnh) – Phần 2.

Kết Luận

Tóm Tắt Kết Quả

Đề án đã triển khai thành công hệ thống nhận diện biển báo giao thông:

1. **Chương trình cơ bản : trafficsign.py** – CNN 2 khối double-Conv (5x5 + 3x3 kernels), ảnh 30×30 , class weights, đạt **99.20% test accuracy** sau 10 epoch. Tự động vẽ và lưu biểu đồ training curves.
2. **Xử lý mất cân bằng** : Phát hiện tỷ lệ mất cân bằng 10:1 (lớp 0: 150 ảnh, lớp 1: 1500 ảnh). Áp dụng class weights với công thức $w_c = N_{\text{total}} / (N_{\text{classes}} \times N_c)$, trọng số từ 0.413 đến 4.130. Các lớp thiểu số (58–210 ảnh) đều đạt F1-score ≥ 0.96 trong Model A.
3. **Giảm learning rate tự động (ReduceLROnPlateau)**: Kỹ thuật này giúp các mô hình hội tụ tốt hơn, đặc biệt là Model C, khi validation loss tiếp tục giảm đều ở giai đoạn cuối huấn luyện, góp phần đạt 99.96% accuracy.
4. **So sánh thuật toán** : Huấn luyện và so sánh 4 mô hình với data augmentation và ReduceLROnPlateau:
 - Model A – Baseline CNN + Class Weights: **99.19%**
 - Model B – CNN + L2 Reg + Class Weights: **99.14%**
 - Model C – Deeper CNN (3 Blocks + BN): **99.96%** (tốt nhất)
 - Model D – MobileNetV2 Transfer Learning: **98.48%**
5. **Công cụ inference**:
 - test_models.py – Dự đoán ảnh đơn với top-3 confidence.
 - test_model_folder.py – Dự đoán thư mục ảnh, đối chiếu ground truth CSV.

Bài Học Kinh Nghiệm

- **BatchNorm + Chiều sâu = Hiệu suất vượt trội**: Model C (99.96%) vượt Model A (99.19%) 0.77%. BatchNorm ổn định hội tụ và cho phép mạng sâu hơn học hiệu quả. Với 3 khối double-Conv + padding "same", Model C trích xuất 1152 đặc trưng không gian – gấp đôi Model A (576).

- **Data augmentation và ReduceLROnPlateau cải thiện độ ổn định:** Việc áp dụng các kỹ thuật này giúp mô hình hội tụ tốt hơn và tránh overfitting.
- **30 epoch là đủ để hội tụ:** Cả 4 mô hình đều hoàn thành 30 epoch với validation accuracy ổn định, ReduceLROnPlateau tự động điều chỉnh learning rate.
- **Class weights hiệu quả:** Dù làm giảm nhẹ accuracy tổng thể, class weights đảm bảo mô hình không bỏ qua các lớp thiểu số – điều quan trọng cho an toàn xe tự lái.
- **L2 Regularization có tác động nhẹ:** Model B (99.14%) chỉ thấp hơn Model A (99.19%) 0.05%. Với Dropout(0.25 + 0.5) đã đủ chống overfitting, thêm L2 ($\lambda = 0.001$) không cải thiện đáng kể.
- **CNN tự thiết kế phù hợp hơn Transfer Learning:** MobileNetV2 (98.48%) thua tất cả CNN tự thiết kế. Với ảnh kích thước nhỏ (30×30) và domain đặc thù, CNN tự thiết kế cho kết quả tốt hơn.

Hướng Phát Triển

- **Tăng kích thước ảnh đầu vào:** Từ 30×30 lên 64×64 hoặc 96×96 để bảo toàn chi tiết chữ số trên biển báo tốc độ – nhóm lớp khó phân biệt nhất.
- **Thử nghiệm thêm kiến trúc:** EfficientNet, ResNet50, hoặc Vision Transformer (ViT) với ảnh độ phân giải cao hơn.
- **Fine-tuning toàn bộ MobileNetV2:** Mở khóa toàn bộ base model sau khi huấn luyện phần đầu để thích nghi tốt hơn với domain biển báo.
- **Triển khai thực tế:** Nhúng model vào ứng dụng di động (TensorFlow Lite) hoặc hệ thống nhúng trên xe. Model C đủ nhẹ để triển khai trên thiết bị biên.
- **Phát hiện biển báo (Detection):** Mở rộng từ classification sang detection bằng YOLO hoặc Faster R-CNN để xác định vị trí biển báo trong ảnh toàn cảnh.
- **Cải thiện inference tools:** Hỗ trợ thêm video input, real-time webcam, và batch processing với nhiều định dạng ground truth.